DECOUPLED WEIGHT DECAY REGULARIZATION

Ilya Loshchilov & Frank Hutter
University of Freiburg
Freiburg, Germany,
{ilya,fh}@cs.uni-freiburg.de

arXiv:1711.05101v3 [cs.LG] 4 Jan 2019

ABSTRACT

L_2 regularization and weight decay regularization are equivalent for standard stochastic gradient descent (when rescaled by the learning rate), but as we demonstrate this is *not* the case for adaptive gradient algorithms, such as Adam. While common implementations of these algorithms employ L_2 regularization (often calling it "weight decay" in what may be misleading due to the inequivalence we expose), we propose a simple modification to recover the original formulation of weight decay regularization by *decoupling* the weight decay from the optimization steps taken w.r.t. the loss function. We provide empirical evidence that our proposed modification (i) decouples the optimal choice of weight decay factor from the setting of the learning rate for both standard SGD and Adam and (ii) substantially improves Adam's generalization performance, allowing it to compete with SGD with momentum on image classification datasets (on which it was previously typically outperformed by the latter). Our proposed decoupled weight decay has already been adopted by many researchers, and the community has implemented it in TensorFlow and PyTorch; the complete source code for our experiments is available at https://github.com/loshchil/AdamW-and-SGDW

1 INTRODUCTION

Adaptive gradient methods, such as AdaGrad (Duchi et al., 2011), RMSProp (Tieleman & Hinton, 2012), Adam (Kingma & Ba, 2014) and most recently AMSGrad (Reddi et al., 2018) have become a default method of choice for training feed-forward and recurrent neural networks (Xu et al., 2015; Radford et al., 2015). Nevertheless, state-of-the-art results for popular image classification datasets, such as CIFAR-10 and CIFAR-100 Krizhevsky (2009), are still obtained by applying SGD with momentum (Gastaldi, 2017; Cubuk et al., 2018). Furthermore, Wilson et al. (2017) suggested that adaptive gradient methods do not generalize as well as SGD with momentum when tested on a diverse set of deep learning tasks, such as image classification, character-level language modeling and constituency parsing. Different hypotheses about the origins of this worse generalization have been investigated, such as the presence of sharp local minima (Keskar et al., 2016; Dinh et al., 2017) and inherent problems of adaptive gradient methods (Wilson et al., 2017). In this paper, we investigate whether it is better to use L_2 regularization or weight decay regularization to train deep neural networks with SGD and Adam. We show that a major factor of the poor generalization of the most popular adaptive gradient method, Adam, is due to the fact that L_2 regularization is not nearly as effective for it as for SGD. Specifically, our analysis of Adam leads to the following observations:

L_2 regularization and weight decay are not identical. The two techniques can be made equivalent for SGD by a reparameterization of the weight decay factor based on the learning rate; however, as is often overlooked, this is not the case for Adam. In particular, when combined with adaptive gradients, L_2 regularization leads to weights with large historic parameter and/or gradient amplitudes being regularized less than they would be when using weight decay.

L_2 regularization is not effective in Adam. One possible explanation why Adam and other adaptive gradient methods might be outperformed by SGD with momentum is that common deep learning libraries only implement L_2 regularization, not the original weight decay. Therefore, on tasks/datasets where the use of L_2 regularization is beneficial for SGD (e.g.,

解耦式权重衰减正则化

Ilya Loshchilov与Frank Hutter
德国弗赖堡大学，弗赖堡，{ilya,fh}
@cs.uni-freiburg.de

arXiv:1711.05101v3 [cs.LG] 4 Jan 2019

摘要

L_2 regularization and weight decay regularization are equivalent for standard stochastic gradient descent (when rescaled by the learning rate), but as we demonstrate this is *not* the case for adaptive gradient algorithms, such as Adam. While common implementations of these algorithms employ L_2 regularization (often calling it "weight decay" in what may be misleading due to the inequivalence we expose), we propose a simple modification to recover the original formulation of weight decay regularization by *decoupling* the weight decay from the optimization steps taken w.r.t. the loss function. We provide empirical evidence that our proposed modification (i) decouples the optimal choice of weight decay factor from the setting of the learning rate for both standard SGD and Adam and (ii) substantially improves Adam's generalization performance, allowing it to compete with SGD with momentum on image classification datasets (on which it was previously typically outperformed by the latter). Our proposed decoupled weight decay has already been adopted by many researchers, and the community has implemented it in TensorFlow and PyTorch; the complete source code for our experiments is available at https://github.com/loshchil/AdamW-and-SGDW

1 引言

Adaptive gradient methods, such as AdaGrad (Duchi et al., 2011), RMSProp (Tieleman & Hinton, 2012), Adam (Kingma & Ba, 2014) and most recently AMSGrad (Reddi et al., 2018) have become a default method of choice for training feed-forward and recurrent neural networks (Xu et al., 2015; Radford et al., 2015). Nevertheless, state-of-the-art results for popular image classification datasets, such as CIFAR-10 and CIFAR-100 Krizhevsky (2009), are still obtained by applying SGD with momentum (Gastaldi, 2017; Cubuk et al., 2018). Furthermore, Wilson et al. (2017) suggested that adaptive gradient methods do not generalize as well as SGD with momentum when tested on a diverse set of deep learning tasks, such as image classification, character-level language modeling and constituency parsing. Different hypotheses about the origins of this worse generalization have been investigated, such as the presence of sharp local minima (Keskar et al., 2016; Dinh et al., 2017) and inherent problems of adaptive gradient methods (Wilson et al., 2017). In this paper, we investigate whether it is better to use L_2 regularization or weight decay regularization to train deep neural networks with SGD and Adam. We show that a major factor of the poor generalization of the most popular adaptive gradient method, Adam, is due to the fact that L_2 regularization is not nearly as effective for it as for SGD. Specifically, our analysis of Adam leads to the following observations:

L_2 权重衰减与正则化并非完全相同。通过根据学习率对权重衰减因子进行重新参数化，这两种技术对于随机梯度下降来说是等价的；然而人们常常忽略的是，Adam优化器并非如此。尤其是当与自适应梯度结合使用时，L正则化会导致那些历史参数值和/或梯度幅值较大的权重，其被正则化的程度会比使用传统权重衰减时更低。

L_2 在Adam优化器中，权重衰减并不起作用。Adam以及其他自适应梯度方法之所以可能被带有动量的随机梯度下降超越，一个可能的解释是：常见的深度学习库仅实现了L_2 正则化，而非原始的权重衰减机制。因此，在那些对随机梯度下降而言使用L_2 正则化能带来益处的任务或数据集上（例如许多流行的图像分类数据集），Adam优化器会得到比带有动量的随机梯度下降更差的结果——而后者中的L正则化则能按预期发挥作用。

on many popular image classification datasets), Adam leads to worse results than SGD with momentum (for which L_2 regularization behaves as expected).

Weight decay is equally effective in both SGD and Adam. For SGD, it is equivalent to L_2 regularization, while for Adam it is not.

Optimal weight decay depends on the total number of batch passes/weight updates. Our empirical analysis of SGD and Adam suggests that the larger the runtime/number of batch passes to be performed, the smaller the optimal weight decay.

Adam can substantially benefit from a scheduled learning rate multiplier. The fact that Adam is an adaptive gradient algorithm and as such adapts the learning rate for each parameter does *not* rule out the possibility to substantially improve its performance by using a global learning rate multiplier, scheduled, e.g., by cosine annealing.

The main contribution of this paper is to *improve regularization in Adam by decoupling the weight decay from the gradient-based update*. In a comprehensive analysis, we show that Adam generalizes substantially better with decoupled weight decay than with L_2 regularization, achieving 15% relative improvement in test error (see Figures 2 and 3); this holds true for various image recognition datasets (CIFAR-10 and ImageNet32x32), training budgets (ranging from 100 to 1800 epochs), and learning rate schedules (fixed, drop-step, and cosine annealing; see Figure 1). We also demonstrate that our decoupled weight decay renders the optimal settings of the learning rate and the weight decay factor much more independent, thereby easing hyperparameter optimization (see Figure 2).

The main motivation of this paper is to improve Adam to make it competitive w.r.t. SGD with momentum even for those problems where it did not use to be competitive. We hope that as a result, practitioners do not need to switch between Adam and SGD anymore, which in turn should reduce the common issue of selecting dataset/task-specific training algorithms and their hyperparameters.

2 Decoupling the Weight Decay from the Gradient-based Update

In the weight decay described by Hanson & Pratt (1988), the weights $\boldsymbol{\theta}$ decay exponentially as

$$\boldsymbol{\theta}_{t+1} = (1-\lambda)\boldsymbol{\theta}_t - \alpha \nabla f_t(\boldsymbol{\theta}_t), \tag{1}$$

where λ defines the rate of the weight decay per step and $\nabla f_t(\boldsymbol{\theta}_t)$ is the t-th batch gradient to be multiplied by a learning rate α. For standard SGD, it is equivalent to standard L_2 regularization:

Proposition 1 (Weight decay = L_2 reg for standard SGD)**.** *Standard SGD with base learning rate α executes the same steps on batch loss functions $f_t(\boldsymbol{\theta})$ with weight decay λ (defined in Equation 1) as it executes without weight decay on $f_t^{reg}(\boldsymbol{\theta}) = f_t(\boldsymbol{\theta}) + \frac{\lambda'}{2}\|\boldsymbol{\theta}\|_2^2$, with $\lambda' = \frac{\lambda}{\alpha}$.*

The proofs of this well-known fact, as well as our other propositions, are given in Appendix A.

Due to this equivalence, L_2 regularization is very frequently referred to as weight decay, including in popular deep learning libraries. However, as we will demonstrate later in this section, this equivalence does *not* hold for adaptive gradient methods. One fact that is often overlooked already for the simple case of SGD is that in order for the equivalence to hold, the L_2 regularizer λ' has to be set to $\frac{\lambda}{\alpha}$, i.e., if there is an overall best weight decay value λ, the best value of λ' is tightly coupled with the learning rate α. In order to decouple the effects of these two hyperparameters, we advocate to decouple the weight decay step as proposed by Hanson & Pratt (1988) (Equation 1).

Looking first at the case of SGD, we propose to decay the weights simultaneously with the update of $\boldsymbol{\theta}_t$ based on gradient information in Line 9 of Algorithm 1. This yields our proposed variant of SGD with momentum using decoupled weight decay (**SGDW**). This simple modification explicitly decouples λ and α (although some problem-dependent implicit coupling may of course remain as for any two hyperparameters). In order to account for a possible scheduling of both α and λ, we introduce a scaling factor η_t delivered by a user-defined procedure $SetScheduleMultiplier(t)$.

Now, let's turn to adaptive gradient algorithms like the popular optimizer Adam Kingma & Ba (2014), which scale gradients by their historic magnitudes. Intuitively, when Adam is run on a loss function f plus L_2 regularization, weights that tend to have large gradients in f do not get regularized as much as they would with decoupled weight decay, since the gradient of the regularizer gets scaled

在许多流行的图像分类数据集上，Adam优化器所带来的结果甚至不如带有动量的随机梯度下降（因为后者中的L_2 正则化能够发挥应有的作用）。

权重衰减在随机梯度下降和Adam优化器中都具有同等的效果。对于随机梯度下降而言，它相当于L_2正则化，而对于Adam优化器则并非如此。

最优的权重衰减值取决于批量迭代次数与参数更新量的总和。我们对随机梯度下降算法和Adam优化器的实证分析表明，所需的运行时间或批量迭代次数越多，最优的权重衰减值就越小。

Adam优化器能够从按计划变化的学习率乘数中获得显著益处。尽管Adam优化器属于自适应梯度算法，会为每个参数单独调整学习率，但这并不排除通过使用全局学习率乘数来大幅提升其性能的可能性，例如通过余弦退火的方式实现按计划变化的学习率。

The main contribution of this paper is to *improve regularization in Adam by decoupling the weight decay from the gradient-based update*. In a comprehensive analysis, we show that Adam generalizes substantially better with decoupled weight decay than with L_2 regularization, achieving 15% relative improvement in test error (see Figures 2 and 3); this holds true for various image recognition datasets (CIFAR-10 and ImageNet32x32), training budgets (ranging from 100 to 1800 epochs), and learning rate schedules (fixed, drop-step, and cosine annealing; see Figure 1). We also demonstrate that our decoupled weight decay renders the optimal settings of the learning rate and the weight decay factor much more independent, thereby easing hyperparameter optimization (see Figure 2).

本文的主要动机在于改进Adam优化器，使其即便在以往无法与带动量的随机梯度下降算法抗衡的问题上，也能具备与之竞争的能力。我们期望由此让从业者无需再在Adam优化器与随机梯度下降算法之间来回切换，进而减少为不同数据集或任务选择特定训练算法及其超参数这一常见难题。

2 基于梯度的更新与权重衰减的解耦

在Hanson与Pratt（1988）所描述的权重衰减机制中，权重 $\boldsymbol{\theta}$ 会呈指数级衰减。

$$\boldsymbol{\theta}_{t+1} = (1-\lambda)\boldsymbol{\theta}_t - \alpha \nabla f_t(\boldsymbol{\theta}_t), \tag{1}$$

其中 λ 定义了每一步的权重衰减速率，而 $\nabla f_t(\boldsymbol{\theta}_t)$则是第 t批梯度，需乘以学习率 α。对于标准的随机梯度下降算法而言，这等价于标准的L_2 正则化方法。

Proposition 1 (Weight decay = L_2 reg for standard SGD)**.** *Standard SGD with base learning rate α executes the same steps on batch loss functions $f_t(\boldsymbol{\theta})$ with weight decay λ (defined in Equation 1) as it executes without weight decay on $f_t^{reg}(\boldsymbol{\theta}) = f_t(\boldsymbol{\theta}) + \frac{\lambda'}{2}\|\boldsymbol{\theta}\|_2^2$, with $\lambda' = \frac{\lambda}{\alpha}$.*

这一著名事实的证明以及我们的其他命题均列于附录A中。

由于这种等价性，L_2 正则化经常被称作权重衰减，就连那些流行的深度学习库也是如此。然而正如我们将在本节后文所示，这种等价性并不适用于自适应梯度方法。即便在简单的随机梯度下降案例中，也有一个常被忽视的事实：要想保持这种等价性，L_2 正则化器 λ' 必须设置为$\frac{\lambda}{\alpha}$，也就是说，如果存在一个全局最优的权重衰减值 λ，那么 λ' 的最佳取值就会与学习率 α紧密关联。为了解耦这两个超参数的影响，我们主张采用Hanson与Pratt（1988）提出的方法——将权重衰减步骤分开处理（方程1）。

首先以随机梯度下降为例，我们提议根据算法1第9行中的梯度信息，在更新 $\boldsymbol{\theta}_t$ 的同时对权重进行衰减。由此便形成了我们所提出的带有动量的随机梯度下降变体，即采用解耦式权重衰减的**SGDW**算法。这一简单的修改明确地将 λ 与 α 解耦开来（当然，与任何两个超参数一样，仍可能存在一些因具体问题而产生的隐性耦合）。为了解决可能需要对 α 和 λ同时进行调度的情况，我们引入了一个由用户自定义的程序 $SetScheduleMultiplier(t)$来生成的缩放因子 η_t 。

现在，让我们来看看诸如广受欢迎的Adam优化器这类自适应梯度算法——它由金格玛与巴在2014年提出，能够根据梯度的历史大小对其数值进行缩放。从直观上讲，当在包含 f plusL_2 正则化的损失函数上运行Adam优化器时，那些在 f 中倾向于产生较大梯度的权重，其正则化程度会低于使用解耦式权重衰减时的情况，因为此时正则化项的梯度会被缩放处理。

Algorithm 1 SGD with L_2 regularization and SGD with decoupled weight decay (SGDW), both with momentum

given initial learning rate $\alpha \in \mathbb{R}$, momentum factor $\beta_1 \in \mathbb{R}$, weight decay/L_2 regularization factor $\lambda \in \mathbb{R}$
initialize time step $t \leftarrow 0$, parameter vector $\boldsymbol{\theta}_{t=0} \in \mathbb{R}^n$, first moment vector $\boldsymbol{m}_{t=0} \leftarrow \boldsymbol{0}$, schedule multiplier $\eta_{t=0} \in \mathbb{R}$
repeat
  $t \leftarrow t + 1$
  $\nabla f_t(\boldsymbol{\theta}_{t-1}) \leftarrow \text{SelectBatch}(\boldsymbol{\theta}_{t-1})$ ▷ select batch and return the corresponding gradient
  $\boldsymbol{g}_t \leftarrow \nabla f_t(\boldsymbol{\theta}_{t-1}) + \lambda \boldsymbol{\theta}_{t-1}$
  $\eta_t \leftarrow \text{SetScheduleMultiplier}(t)$ ▷ can be fixed, decay, be used for warm restarts
  $\boldsymbol{m}_t \leftarrow \beta_1 \boldsymbol{m}_{t-1} + \eta_t \alpha \boldsymbol{g}_t$
  $\boldsymbol{\theta}_t \leftarrow \boldsymbol{\theta}_{t-1} - \boldsymbol{m}_t - \eta_t \lambda \boldsymbol{\theta}_{t-1}$
until *stopping criterion is met*
return optimized parameters $\boldsymbol{\theta}_t$

Algorithm 2 Adam with L_2 regularization and Adam with decoupled weight decay (AdamW)

given $\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}, \lambda \in \mathbb{R}$
initialize time step $t \leftarrow 0$, parameter vector $\boldsymbol{\theta}_{t=0} \in \mathbb{R}^n$, first moment vector $\boldsymbol{m}_{t=0} \leftarrow \boldsymbol{0}$, second moment vector $\boldsymbol{v}_{t=0} \leftarrow \boldsymbol{0}$, schedule multiplier $\eta_{t=0} \in \mathbb{R}$
repeat
  $t \leftarrow t + 1$
  $\nabla f_t(\boldsymbol{\theta}_{t-1}) \leftarrow \text{SelectBatch}(\boldsymbol{\theta}_{t-1})$ ▷ select batch and return the corresponding gradient
  $\boldsymbol{g}_t \leftarrow \nabla f_t(\boldsymbol{\theta}_{t-1}) + \lambda \boldsymbol{\theta}_{t-1}$
  $\boldsymbol{m}_t \leftarrow \beta_1 \boldsymbol{m}_{t-1} + (1 - \beta_1) \boldsymbol{g}_t$ ▷ here and below all operations are element-wise
  $\boldsymbol{v}_t \leftarrow \beta_2 \boldsymbol{v}_{t-1} + (1 - \beta_2) \boldsymbol{g}_t^2$
  $\hat{\boldsymbol{m}}_t \leftarrow \boldsymbol{m}_t / (1 - \beta_1^t)$ ▷ β_1 is taken to the power of t
  $\hat{\boldsymbol{v}}_t \leftarrow \boldsymbol{v}_t / (1 - \beta_2^t)$ ▷ β_2 is taken to the power of t
  $\eta_t \leftarrow \text{SetScheduleMultiplier}(t)$ ▷ can be fixed, decay, or also be used for warm restarts
  $\boldsymbol{\theta}_t \leftarrow \boldsymbol{\theta}_{t-1} - \eta_t \left(\alpha \hat{\boldsymbol{m}}_t / (\sqrt{\hat{\boldsymbol{v}}_t} + \epsilon) + \lambda \boldsymbol{\theta}_{t-1} \right)$
until *stopping criterion is met*
return optimized parameters $\boldsymbol{\theta}_t$

along with the gradient of f. This leads to an inequivalence of L_2 and decoupled weight decay regularization for adaptive gradient algorithms:

Proposition 2 (Weight decay $\neq$ L_2 reg for adaptive gradients)**.** *Let O denote an optimizer that has iterates $\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \alpha \mathbf{M}_t \nabla f_t(\boldsymbol{\theta}_t)$ when run on batch loss function $f_t(\boldsymbol{\theta})$* without *weight decay, and $\boldsymbol{\theta}_{t+1} \leftarrow (1 - \lambda)\boldsymbol{\theta}_t - \alpha \mathbf{M}_t \nabla f_t(\boldsymbol{\theta}_t)$ when run on $f_t(\boldsymbol{\theta})$* with *weight decay, respectively, with $\mathbf{M}_t \neq k\mathbf{I}$ (where $k \in \mathbb{R}$). Then, for O there exists no L_2 coefficient λ' such that running O on batch loss $f_t^{reg}(\boldsymbol{\theta}) = f_t(\boldsymbol{\theta}) + \frac{\lambda'}{2} \|\boldsymbol{\theta}\|_2^2$ without weight decay is equivalent to running O on $f_t(\boldsymbol{\theta})$ with decay $\lambda \in \mathbb{R}^+$.*

We decouple weight decay and loss-based gradient updates in Adam as shown in line 12 of Algorithm 2; this gives rise to our variant of Adam with decoupled weight decay (**AdamW**).

Having shown that L_2 regularization and weight decay regularization differ for adaptive gradient algorithms raises the question of how they differ and how to interpret their effects. Their equivalence for standard SGD remains very helpful for intuition: both mechanisms push weights closer to zero, at the same rate. However, for adaptive gradient algorithms they differ: with L_2 regularization, the sums of the gradient of the loss function and the gradient of the regularizer (i.e., the L_2 norm of the weights) are adapted, whereas with decoupled weight decay, only the gradients of the loss function are adapted (with the weight decay step separated from the adaptive gradient mechanism). With L_2 regularization both types of gradients are normalized by their typical (summed) magnitudes, and therefore weights x with large typical gradient magnitude s are regularized by a smaller relative amount than other weights. In contrast, decoupled weight decay regularizes all weights with the same rate λ, effectively regularizing weights x with large s more than standard L_2 regularization

Algorithm 1 SGD with L_2 regularization and SGD with decoupled weight decay (SGDW), both with momentum

given initial learning rate $\alpha \in \mathbb{R}$, momentum factor $\beta_1 \in \mathbb{R}$, weight decay/L_2 regularization factor $\lambda \in \mathbb{R}$
initialize time step $t \leftarrow 0$, parameter vector $\boldsymbol{\theta}_{t=0} \in \mathbb{R}^n$, first moment vector $\boldsymbol{m}_{t=0} \leftarrow \boldsymbol{0}$, schedule multiplier $\eta_{t=0} \in \mathbb{R}$
repeat
  $t \leftarrow t + 1$
  $\nabla f_t(\boldsymbol{\theta}_{t-1}) \leftarrow \text{SelectBatch}(\boldsymbol{\theta}_{t-1})$ ▷ select batch and return the corresponding gradient
  $\boldsymbol{g}_t \leftarrow \nabla f_t(\boldsymbol{\theta}_{t-1}) + \lambda \boldsymbol{\theta}_{t-1}$
  $\eta_t \leftarrow \text{SetScheduleMultiplier}(t)$ ▷ can be fixed, decay, be used for warm restarts
  $\boldsymbol{m}_t \leftarrow \beta_1 \boldsymbol{m}_{t-1} + \eta_t \alpha \boldsymbol{g}_t$
  $\boldsymbol{\theta}_t \leftarrow \boldsymbol{\theta}_{t-1} - \boldsymbol{m}_t - \eta_t \lambda \boldsymbol{\theta}_{t-1}$
until *stopping criterion is met*
return optimized parameters $\boldsymbol{\theta}_t$

算法2带L2正 具有)

given $\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}, \lambda \in \mathbb{R}$
initialize time step $t \leftarrow 0$, parameter vector $\boldsymbol{\theta}_{t=0} \in \mathbb{R}^n$, first moment vector $\boldsymbol{m}_{t=0} \leftarrow \boldsymbol{0}$, second moment vector $\boldsymbol{v}_{t=0} \leftarrow \boldsymbol{0}$, schedule multiplier $\eta_{t=0} \in \mathbb{R}$
repeat
  $t \leftarrow t + 1$
  $\nabla f_t(\boldsymbol{\theta}_{t-1}) \leftarrow \text{SelectBatch}(\boldsymbol{\theta}_{t-1})$ ▷ select batch and return the corresponding gradient
  $\boldsymbol{g}_t \leftarrow \nabla f_t(\boldsymbol{\theta}_{t-1}) + \lambda \boldsymbol{\theta}_{t-1}$
  $\boldsymbol{m}_t \leftarrow \beta_1 \boldsymbol{m}_{t-1} + (1 - \beta_1) \boldsymbol{g}_t$ ▷ here and below all operations are element-wise
  $\boldsymbol{v}_t \leftarrow \beta_2 \boldsymbol{v}_{t-1} + (1 - \beta_2) \boldsymbol{g}_t^2$
  $\hat{\boldsymbol{m}}_t \leftarrow \boldsymbol{m}_t / (1 - \beta_1^t)$ ▷ β_1 is taken to the power of t
  $\hat{\boldsymbol{v}}_t \leftarrow \boldsymbol{v}_t / (1 - \beta_2^t)$ ▷ β_2 is taken to the power of t
  $\eta_t \leftarrow \text{SetScheduleMultiplier}(t)$ ▷ can be fixed, decay, or also be used for warm restarts
  $\boldsymbol{\theta}_t \leftarrow \boldsymbol{\theta}_{t-1} - \eta_t \left(\alpha \hat{\boldsymbol{m}}_t / (\sqrt{\hat{\boldsymbol{v}}_t} + \epsilon) + \lambda \boldsymbol{\theta}_{t-1} \right)$
until *stopping criterion is met*
return optimized parameters $\boldsymbol{\theta}_t$

以及 f的梯度。这就导致了自适应梯度算法中的L_2 与权重衰减正则化之间存在不等价性：

命题*2*（针对自适应梯度的权重衰减 $\neq$ L_2 *reg*）。设 O 表示一个优化器：在没有权重衰减的情况下，它在批量损失函数 $f_t(\boldsymbol{\theta})$上运行时会进行 $\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \alpha M_t \nabla f_t(\boldsymbol{\theta}_t)$次迭代；而在有权重衰减的情况下，它在 $f_t(\boldsymbol{\theta})$上运行时则会进行 $\boldsymbol{\theta}_{t+1} \leftarrow (1 - \lambda)\boldsymbol{\theta}_t - \alpha M_t \nabla f_t(\boldsymbol{\theta}_t)$次迭代，此处$M_t \neq k$ I（其中 $k \in \mathbb{R}$）。那么，对于 O 而言，不存在这样的L_2 系数 λ'，使得在没有权重衰减的批量损失 $f_t^{reg}(\boldsymbol{\theta}) = f_t(\boldsymbol{\theta}) + \frac{\lambda'}{2} \|\boldsymbol{\theta}\|_2^2$ 上运行 O，能够等同于在带有衰减$\lambda \in \mathbb{R}^+$的 $f_t(\boldsymbol{\theta})$上运行 O。

We 如算法第12行所示，Adam优化器中实现了权重衰减与基于损失函数的梯度参数更新量的解耦。- rithm 2；这一设计便构成了我们提出的带有权重衰减解耦机制的Adam优化器变体（**AdamW**）。

Having shown that L_2 regularization and weight decay regularization differ for adaptive gradient algorithms raises the question of how they differ and how to interpret their effects. Their equivalence for standard SGD remains very helpful for intuition: both mechanisms push weights closer to zero, at the same rate. However, for adaptive gradient algorithms they differ: with L_2 regularization, the sums of the gradient of the loss function and the gradient of the regularizer (i.e., the L_2 norm of the weights) are adapted, whereas with decoupled weight decay, only the gradients of the loss function are adapted (with the weight decay step separated from the adaptive gradient mechanism). With L_2 regularization both types of gradients are normalized by their typical (summed) magnitudes, and therefore weights x with large typical gradient magnitude s are regularized by a smaller relative amount than other weights. In contrast, decoupled weight decay regularizes all weights with the same rate λ, effectively regularizing weights x with large s more than standard L_2 regularization

does. We demonstrate this formally for a simple special case of adaptive gradient algorithm with a fixed preconditioner:

Proposition 3 (Weight decay = scale-adjusted L_2 reg for adaptive gradient algorithm with fixed preconditioner)**.** *Let O denote an algorithm with the same characteristics as in Proposition 2, and using a fixed preconditioner matrix $\boldsymbol{M}_t = diag(\boldsymbol{s})^{-1}$ (with $s_i > 0$ for all i). Then, O with base learning rate α executes the same steps on batch loss functions $f_t(\boldsymbol{\theta})$ with weight decay λ as it executes without weight decay on the scale-adjusted regularized batch loss*

$$f_t^{sreg}(\boldsymbol{\theta}) = f_t(\boldsymbol{\theta}) + \frac{\lambda'}{2\alpha} \left\| \boldsymbol{\theta} \odot \sqrt{\boldsymbol{s}} \right\|_2^2, \tag{2}$$

where $\odot$ and $\sqrt{\cdot}$ denote element-wise multiplication and square root, respectively, and $\lambda' = \frac{\lambda}{\alpha}$.

We note that this proposition does *not* directly apply to practical adaptive gradient algorithms, since these change the preconditioner matrix at every step. Nevertheless, it can still provide intuition about the equivalent loss function being optimized in each step: parameters θ_i with a large inverse preconditioner s_i (which in practice would be caused by historically large gradients in dimension i) are regularized relatively more than they would be with L_2 regularization; specifically, the regularization is proportional to $\sqrt{s_i}$.

3 Justification of Decoupled Weight Decay via a View of Adaptive Gradient Methods as Bayesian Filtering

We now discuss a justification of decoupled weight decay in the framework of Bayesian filtering for a unified theory of adaptive gradient algorithms due to Aitchison (2018). After we posted a preliminary version of our current paper on arXiv, Aitchison noted that his theory "gives us a theoretical framework in which we can understand the superiority of this weight decay over L_2 regularization, because it is weight decay, rather than L_2 regularization that emerges through the straightforward application of Bayesian filtering."(Aitchison, 2018). While full credit for this theory goes to Aitchison, we summarize it here to shed some light on why weight decay may be favored over L_2 regularization.

Aitchison (2018) views stochastic optimization of n parameters $\theta_1, \ldots, \theta_n$ as a Bayesian filtering problem with the goal of inferring a distribution over the optimal values of each of the parameters θ_i given the current values of the other parameters $\boldsymbol{\theta}_{-i}(t)$ at time step t. When the other parameters do not change this is an optimization problem, but when they do change it becomes one of "tracking" the optimizer using Bayesian filtering as follows. One is given a probability distribution $P(\boldsymbol{\theta}_t \mid \boldsymbol{y_{1:t}})$ of the optimizer at time step t that takes into account the data $\boldsymbol{y_{1:t}}$ from the first t mini batches, a state transition prior $P(\boldsymbol{\theta}_{t+1} \mid \boldsymbol{\theta}_t)$ reflecting a (small) data-independent change in this distribution from one step to the next, and a likelihood $P(\boldsymbol{y}_{t+1} \mid \boldsymbol{\theta}_{t+1})$ derived from the mini batch at step $t+1$. The posterior distribution $P(\boldsymbol{\theta}_{t+1} \mid \boldsymbol{y_{1:t+1}})$ of the optimizer at time step $t+1$ can then be computed (as usual in Bayesian filtering) by marginalizing over $\boldsymbol{\theta}_t$ to obtain the one-step ahead predictions $P(\boldsymbol{\theta}_{t+1} \mid \boldsymbol{y_{1:t}})$ and then applying Bayes' rule to incorporate the likelihood $P(\boldsymbol{y}_{t+1} \mid \boldsymbol{\theta}_{t+1})$. Aitchison (2018) assumes a Gaussian state transition distribution $P(\boldsymbol{\theta}_{t+1} \mid \boldsymbol{\theta}_t)$ and an approximate conjugate likelihood $P(\boldsymbol{y}_{t+1} \mid \boldsymbol{\theta}_{t+1})$, leading to the following closed-form update of the filtering distribution's mean:

$$\boldsymbol{\mu}_{post} = \boldsymbol{\mu}_{prior} + \boldsymbol{\Sigma}_{post} \times \boldsymbol{g}, \tag{3}$$

where $\boldsymbol{g}$ is the gradient of the log likelihood of the mini batch at time t. This result implies a preconditioner of the gradients that is given by the posterior uncertainty $\boldsymbol{\Sigma}_{post}$ of the filtering distribution: updates are larger for parameters we are more uncertain about and smaller for parameters we are more certain about. Aitchison (2018) goes on to show that popular adaptive gradient methods, such as Adam and RMSprop, as well as Kronecker-factorized methods are special cases of this framework.

Decoupled weight decay very naturally fits into this unified framework as part of the state-transition distribution: Aitchison (2018) assumes a slow change of the optimizer according to the following Gaussian:

$$P(\boldsymbol{\theta}_{t+1} \mid \boldsymbol{\theta}_t) = \mathcal{N}((\boldsymbol{I} - \boldsymbol{A})\boldsymbol{\theta}_t, \boldsymbol{Q}), \tag{4}$$

确实如此。我们针对带有固定先验条件的自适应梯度算法的一个简单特例，对此进行了正式的证明。

Proposition 3 (Weight decay = scale-adjusted L_2 reg for adaptive gradient algorithm with fixed preconditioner)**.** *Let O denote an algorithm with the same characteristics as in Proposition 2, and using a fixed preconditioner matrix $\boldsymbol{M}_t = diag(\boldsymbol{s})^{-1}$ (with $s_i > 0$ for all i). Then, O with base learning rate α executes the same steps on batch loss functions $f_t(\boldsymbol{\theta})$ with weight decay λ as it executes without weight decay on the scale-adjusted regularized batch loss*

$$f_t^{sreg}(\boldsymbol{\theta}) = f_t(\boldsymbol{\theta}) + \frac{\lambda'}{2\alpha} \left\| \boldsymbol{\theta} \odot \sqrt{\boldsymbol{s}} \right\|_2^2, \tag{2}$$

其中 $\odot$ 表示逐元素相乘， $\sqrt{\cdot}$ 表示取平方根，而 $\lambda' = \frac{\lambda}{\alpha}$ 则表示..... –

我们需要指出，这一命题并不直接适用于实际中的自适应梯度算法，因为这类算法会在每一步都改变先验条件矩阵。尽管如此，它依然有助于我们理解在每一步中实际上正在优化的等价损失函数：那些具有较大逆先验条件矩阵 s_i 的参数 θ_i （在实际情况中，这通常是由维度 i上的梯度值长期较大所导致的），其正则化程度会比采用L_2 正则化时的程度更高；具体而言，这种正则化强度与$\sqrt{s_i}$成正比。

3 从自适应梯度方法作为贝叶斯滤波器的视角论证解耦权重衰减的合理性

接下来，我们将在贝叶斯滤波框架下探讨解耦式权重衰减的合理性，以此为艾奇森（2018）提出的自适应梯度算法统一理论提供支撑。在我们将当前论文的初步版本发布到arXiv预印本平台后，艾奇森指出，他的理论“为我们提供了一个理论框架，使我们能够理解为何这种权重衰减比 L_2 正则化更具优势，因为正是通过直接应用贝叶斯滤波而产生的才是权重衰减，而非 L_2 正则化。”（艾奇森，2018）。尽管这一理论的完整功绩应归于艾奇森，但我们在此对其加以总结，旨在阐明为何权重衰减会优于 L_2 正则化。

Aitchison (2018) views stochastic optimization of n parameters $\theta_1, \ldots, \theta_n$ as a Bayesian filtering problem with the goal of inferring a distribution over the optimal values of each of the parameters θ_i given the current values of the other parameters $\boldsymbol{\theta}_{-i}(t)$ at time step t. When the other parameters do not change this is an optimization problem, but when they do change it becomes one of "tracking" the optimizer using Bayesian filtering as follows. One is given a probability distribution $P(\boldsymbol{\theta}_t \mid \boldsymbol{y_{1:t}})$ of the optimizer at time step t that takes into account the data $\boldsymbol{y_{1:t}}$ from the first t mini batches, a state transition prior $P(\boldsymbol{\theta}_{t+1} \mid \boldsymbol{\theta}_t)$ reflecting a (small) data-independent change in this distribution from one step to the next, and a likelihood $P(\boldsymbol{y}_{t+1} \mid \boldsymbol{\theta}_{t+1})$ derived from the mini batch at step $t+1$. The posterior distribution $P(\boldsymbol{\theta}_{t+1} \mid \boldsymbol{y_{1:t+1}})$ of the optimizer at time step $t+1$ can then be computed (as usual in Bayesian filtering) by marginalizing over $\boldsymbol{\theta}_t$ to obtain the one-step ahead predictions $P(\boldsymbol{\theta}_{t+1} \mid \boldsymbol{y_{1:t}})$ and then applying Bayes' rule to incorporate the likelihood $P(\boldsymbol{y}_{t+1} \mid \boldsymbol{\theta}_{t+1})$. Aitchison (2018) assumes a Gaussian state transition distribution $P(\boldsymbol{\theta}_{t+1} \mid \boldsymbol{\theta}_t)$ and an approximate conjugate likelihood $P(\boldsymbol{y}_{t+1} \mid \boldsymbol{\theta}_{t+1})$, leading to the following closed-form update of the filtering distribution's mean:

$$\boldsymbol{\mu}_{post} = \boldsymbol{\mu}_{prior} + \boldsymbol{\Sigma}_{post} \times \boldsymbol{g}, \tag{3}$$

其中 $\boldsymbol{g}$ 表示在时间 t时小批量数据的对数似然的梯度。这一结果意味着梯度的先决条件由滤波分布的后验不确定性 $\boldsymbol{\Sigma}_{post}$ 决定：对于我们不确定性更高的参数，其参数更新量会更大；而对于我们确定性更高的参数，参数更新量则会更小。Aitchison（2018）进一步指出，诸如Adam优化器、RMSprop算法以及克罗内克分解法这类常用的自适应梯度方法，其实都是该框架下的特例。

作为状态转移分布的一部分，解耦式权重衰减能够非常自然地融入这一统一框架：Aitchison（2018）假设优化器的变化遵循如下高斯分布：

$$P(\boldsymbol{\theta}_{t+1} \mid \boldsymbol{\theta}_t) = \mathcal{N}((\boldsymbol{I} - \boldsymbol{A})\boldsymbol{\theta}_t, \boldsymbol{Q}), \tag{4}$$

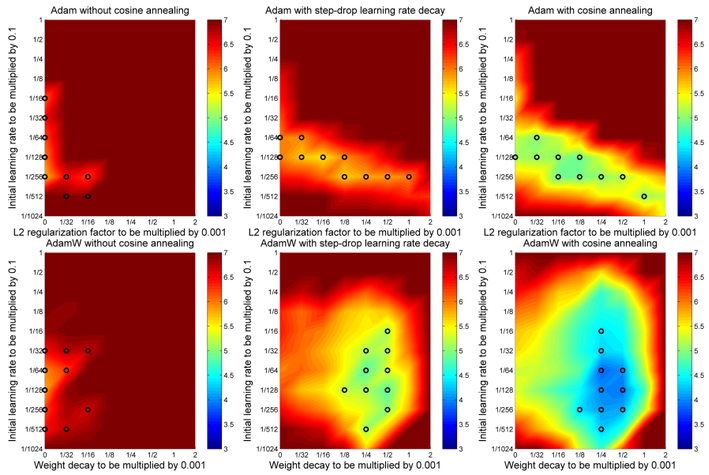


Figure 1: Adam performs better with decoupled weight decay (bottom row, AdamW) than with L_2 regularization (top row, Adam). We show the final test error of a 26 2x64d ResNet on CIFAR-10 after 100 epochs of training with fixed learning rate (left column), step-drop learning rate (with drops at epoch indexes 30, 60 and 80, middle column) and cosine annealing (right column). AdamW leads to a more separable hyperparameter search space, especially when a learning rate schedule, such as step-drop and cosine annealing is applied. Cosine annealing yields clearly superior results.

where $\boldsymbol{Q}$ is the covariance of Gaussian perturbations of the weights, and $\boldsymbol{A}$ is a regularizer to avoid values growing unboundedly over time. When instantiated as $\boldsymbol{A} = \lambda \times \boldsymbol{I}$, this regularizer $\boldsymbol{A}$ plays exactly the role of decoupled weight decay as described in Equation 1, since this leads to multiplying the current mean estimate $\boldsymbol{\theta}_t$ by $(1 - \lambda)$ at each step. Notably, this regularization is also directly applied to the prior and does not depend on the uncertainty in each of the parameters (which would be required for L_2 regularization).

4 Experimental Validation

We now evaluate the performance of decoupled weight decay under various training budgets and learning rate schedules. Our experimental setup follows that of Gastaldi (2017), who proposed, in addition to L_2 regularization, to apply the new Shake-Shake regularization to a 3-branch residual DNN that allowed to achieve new state-of-the-art results of 2.86% on the CIFAR-10 dataset (Krizhevsky, 2009). We used the same model/source code based on fb.resnet.torch [1]. We always used a batch size of 128 and applied the regular data augmentation procedure for the CIFAR datasets. The base networks are a 26 2x64d ResNet (i.e. the network has a depth of 26, 2 residual branches and the first residual block has a width of 64) and a 26 2x96d ResNet with 11.6M and 25.6M parameters, respectively. For a detailed description of the network and the Shake-Shake method, we refer the interested reader to Gastaldi (2017). We also perform experiments on the ImageNet32x32 dataset (Chrabaszcz et al., 2017), a downsampled version of the original ImageNet dataset with 1.2 million 32×32 pixels images.

4.1 Evaluating Decoupled Weight Decay With Different Learning Rate Schedules

In our first experiment, we compare Adam with L_2 regularization to Adam with decoupled weight decay (AdamW), using three different learning rate schedules: a fixed learning rate, a drop-step

[1] https://github.com/xgastaldi/shake-shake

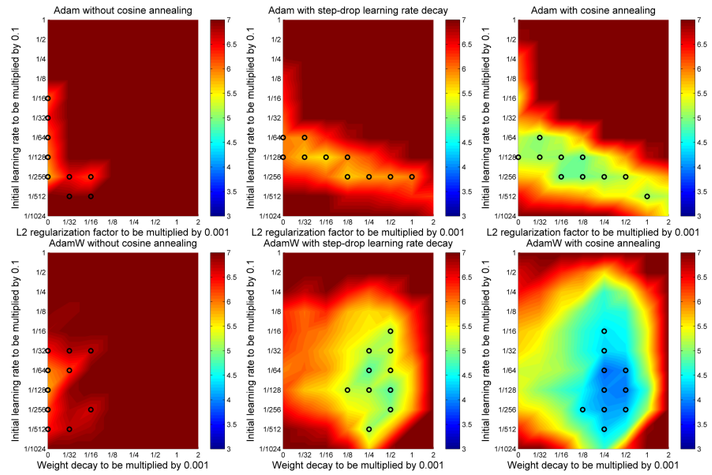


图1：在采用解耦式权重衰减策略时（下排，AdamW），Adam优化器的表现优于 L_2正则化策略（上排，Adam）。我们展示了在固定学习率、阶跃式下降学习率（在训练轮数30、60和80处进行下降处理，中间列）以及余弦退火三种训练策略下，经过100个训练轮数后、在CIFAR-10数据集上使用的26层2x64维ResNet网络的最终测试误差。AdamW能够带来更易于划分的超参数搜索空间，尤其是在应用了阶跃式下降学习率或余弦退火等学习率调度策略时。而余弦退火策略则能产生明显更优的实验结果。

其中 $\boldsymbol{Q}$ 表示权重高斯扰动的协方差， $\boldsymbol{A}$ 则是用于防止数值随时间无限增长的正则化项。当以 $\boldsymbol{A} = \lambda \times \boldsymbol{I}$的形式实现时，这个正则化项 $\boldsymbol{A}$ 恰好能发挥方程1中所描述的解耦式权重衰减的作用，因为这样会在每一步中将当前的均值估计 $\boldsymbol{\theta}_t$ 乘以$(1 - \lambda)$。值得注意的是，这种正则化处理也会直接应用于先验分布，且不依赖于各个参数的不确定性（而 L_2 正则化则需要考虑这一点）。

4 实验验证

We now evaluate the performance of decoupled weight decay under various training budgets and learning rate schedules. Our experimental setup follows that of Gastaldi (2017), who proposed, in addition to L_2 regularization, to apply the new Shake-Shake regularization to a 3-branch residual DNN that allowed to achieve new state-of-the-art results of 2.86% on the CIFAR-10 dataset (Krizhevsky, 2009). We used the same model/source code based on fb.resnet.torch [1]. We always used a batch size of 128 and applied the regular data augmentation procedure for the CIFAR datasets. The base networks are a 26 2x64d ResNet (i.e. the network has a depth of 26, 2 residual branches and the first residual block has a width of 64) and a 26 2x96d ResNet with 11.6M and 25.6M parameters, respectively. For a detailed description of the network and the Shake-Shake method, we refer the interested reader to Gastaldi (2017). We also perform experiments on the ImageNet32x32 dataset (Chrabaszcz et al., 2017), a downsampled version of the original ImageNet dataset with 1.2 million 32×32 pixels images.

4.1 使用不同的学习率调度策略评估解耦权重衰减

在首个实验中，我们对比了采用 L_2 正则化的Adam优化器与采用解耦权重衰减（AdamW算法）的Adam优化器，同时使用了三种不同的学习率调度策略：固定学习率、下降步长法

[1] https://github.com/xgastaldi/shake-shake

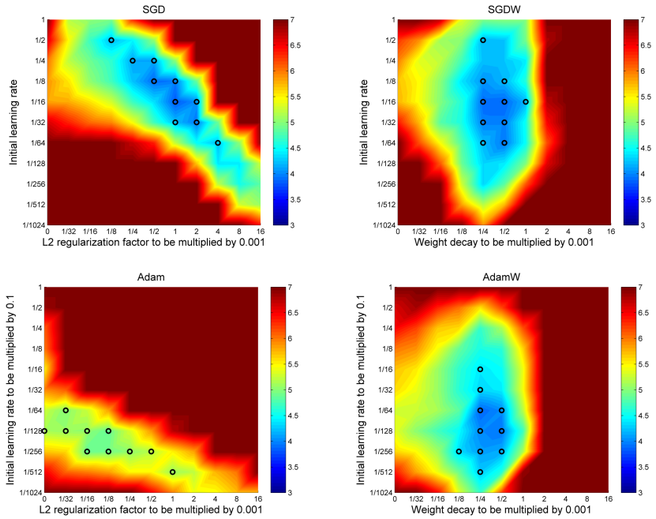


Figure 2: The Top-1 test error of a 26 2x64d ResNet on CIFAR-10 measured after 100 epochs. The proposed SGDW and AdamW (right column) have a more separable hyperparameter space.

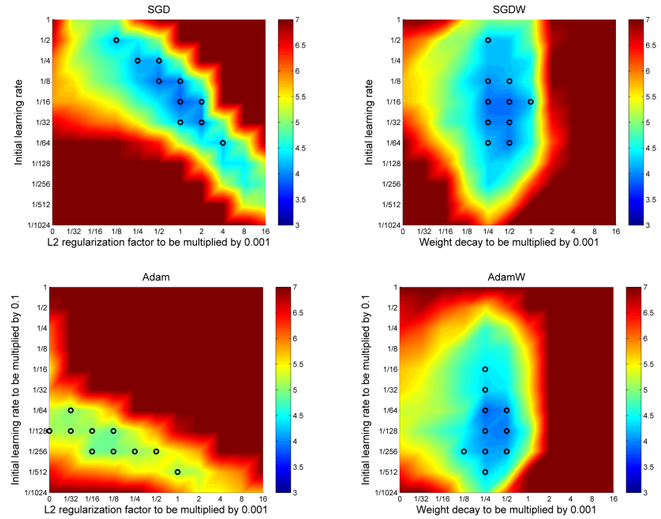
schedule, and a cosine annealing schedule (Loshchilov & Hutter, 2016). Since Adam already adapts its parameterwise learning rates it is not as common to use a learning rate multiplier schedule with it as it is with SGD, but as our results show such schedules can substantially improve Adam's performance, and we advocate not to overlook their use for adaptive gradient algorithms.

For each learning rate schedule and weight decay variant, we trained a 2x64d ResNet for 100 epochs, using different settings of the initial learning rate α and the weight decay factor λ. Figure 1 shows that decoupled weight decay outperforms L_2 regularization for all learning rate schedules, with larger differences for better learning rate schedules. We also note that decoupled weight decay leads to a more separable hyperparameter search space, especially when a learning rate schedule, such as step-drop and cosine annealing is applied. The figure also shows that cosine annealing clearly outperforms the other learning rate schedules; we thus used cosine annealing for the remainder of the experiments.

4.2 Decoupling the Weight Decay and Initial Learning Rate Parameters

In order to verify our hypothesis about the coupling of α and λ, in Figure 2 we compare the performance of L_2 regularization vs. decoupled weight decay in SGD (SGD vs. SGDW, top row) and in Adam (Adam vs. AdamW, bottom row). In SGD (Figure 2, top left), L_2 regularization is not decoupled from the learning rate (the common way as described in Algorithm 1), and the figure clearly shows that the basin of best hyperparameter settings (depicted by color and top-10 hyperparameter settings by black circles) is not aligned with the x-axis or y-axis but lies on the diagonal. This suggests that the two hyperparameters are interdependent and need to be changed simultaneously, while only changing one of them might substantially worsen results. Consider, e.g., the setting at the top left black circle ($\alpha = 1/2$, $\lambda = 1/8 * 0.001$); only changing either α or λ by itself would worsen results, while changing both of them could still yield clear improvements. We note that this coupling of initial learning rate and L_2 regularization factor might have contributed to SGD's reputation of being very sensitive to its hyperparameter settings.

In contrast, the results for SGD with decoupled weight decay (SGDW) in Figure 2 (top right) show that weight decay and initial learning rate are decoupled. The proposed approach renders the two hyperparameters more separable: even if the learning rate is not well tuned yet (e.g., consider the value of 1/1024 in Figure 2, top right), leaving it fixed and only optimizing the weight decay factor



Figu第2行：在完成100个训练轮数后，a262x64d ResNet网络在CIFAR-10数据集上的Top-1测试误差。T
该提议
右列中的SGDW和AdamW算法具有更具可分离性的超参数空间。

正如实验结果所显示的，尽管Adam优化器具备自适应调整参数学习率的特性，使得人们通常不会像使用随机梯度下降那样为其配置学习率乘数调度策略，但此类调度依然能够大幅改善Adam优化器的性能。因此，对于自适应梯度算法而言，我们强烈建议不要忽略这类调度策略的应用价值。

对于每一种学习率调度方案和权重衰减变体，我们均使用初始学习率 α 与权重衰减因子 λ的不同设定，对2x64d规模的ResNet网络进行了100个训练轮数的训练。如图1所示，在所有学习率调度方案下，解耦式权重衰减的性能均优于 L_2 正则化方法，且在学习率更优的调度方案中，两者的性能差距更为显著。我们还发现，解耦式权重衰减能够形成更具可分离性的超参数搜索空间，尤其是在采用了步长下降或余弦退火等学习率调度方案时。该图表同时表明余弦退火的性能明显优于其他学习率调度方案；因此，在后续实验中我们均采用了余弦退火策略。

4.2 实现权重衰减与初始学习率参数的解耦

In order to verify our hypothesis about the coupling of α and λ, in Figure 2 we compare the performance of L_2 regularization vs. decoupled weight decay in SGD (SGD vs. SGDW, top row) and in Adam (Adam vs. AdamW, bottom row). In SGD (Figure 2, top left), L_2 regularization is not decoupled from the learning rate (the common way as described in Algorithm 1), and the figure clearly shows that the basin of best hyperparameter settings (depicted by color and top-10 hyperparameter settings by black circles) is not aligned with the x-axis or y-axis but lies on the diagonal. This suggests that the two hyperparameters are interdependent and need to be changed simultaneously, while only changing one of them might substantially worsen results. Consider, e.g., the setting at the top left black circle ($\alpha = 1/2$, $\lambda = 1/8 * 0.001$); only changing either α or λ by itself would worsen results, while changing both of them could still yield clear improvements. We note that this coupling of initial learning rate and L_2 regularization factor might have contributed to SGD's reputation of being very sensitive to its hyperparameter settings.

相比之下，图2右上角展示的采用SGDW算法并应用分离式权重衰减的随机梯度下降结果表明，权重衰减与初始学习率是相互独立的。所提出的方法使得这两个超参数更具可分离性：即便学习率尚未得到恰当调整（例如图2右上角所示的1/1024这一数值），只需将其保持不变，仅优化权重衰减系数，就能获得良好的效果（即1/4*0.001这样的数值）。而采用L{v1}正则化的随机梯度下降则不存在这种情况（见图2左上角）。

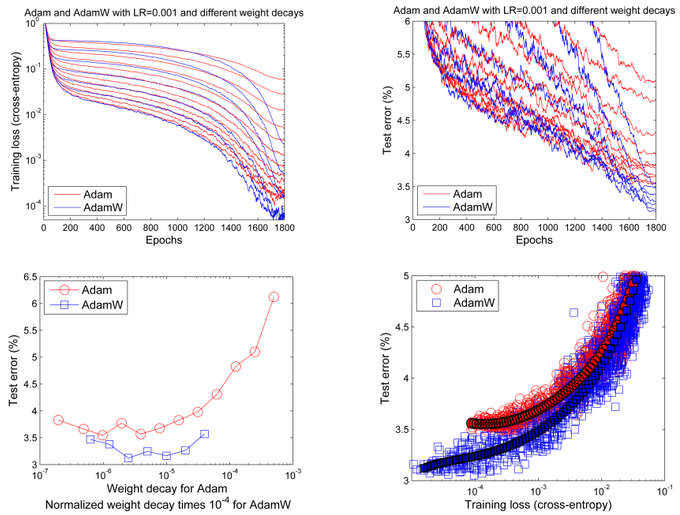


Figure 3: Learning curves (top row) and generalization results (bottom row) obtained by a 26 2x96d ResNet trained with Adam and AdamW on CIFAR-10. See text for details. SuppFigure 4 in the Appendix shows the same qualitative results for ImageNet32x32.

would yield a good value (of 1/4*0.001). This is not the case for SGD with L_2 regularization (see Figure 2, top left).

The results for Adam with L_2 regularization are given in Figure 2 (bottom left). Adam's best hyperparameter settings performed clearly worse than SGD's best ones (compare Figure 2, top left). While both methods used L_2 regularization, Adam did not benefit from it at all: its best results obtained for non-zero L_2 regularization factors were comparable to the best ones obtained without the L_2 regularization, i.e., when $\lambda = 0$. Similarly to the original SGD, the shape of the hyperparameter landscape suggests that the two hyperparameters are coupled.

In contrast, the results for our new variant of Adam with decoupled weight decay (AdamW) in Figure 2 (bottom right) show that AdamW largely decouples weight decay and learning rate. The results for the best hyperparameter settings were substantially better than the best ones of Adam with L_2 regularization and rivaled those of SGD and SGDW.

In summary, the results in Figure 2 support our hypothesis that the weight decay and learning rate hyperparameters can be decoupled, and that this in turn simplifies the problem of hyperparameter tuning in SGD and improves Adam's performance to be competitive w.r.t. SGD with momentum.

4.3 Better Generalization of AdamW

While the previous experiment suggested that the basin of optimal hyperparameters of AdamW is broader and deeper than the one of Adam, we next investigated the results for much longer runs of 1800 epochs to compare the generalization capabilities of AdamW and Adam.

We fixed the initial learning rate to 0.001 which represents both the default learning rate for Adam and the one which showed reasonably good results in our experiments. Figure 3 shows the results for 12 settings of the L_2 regularization of Adam and 7 settings of the normalized weight decay of AdamW (the normalized weight decay represents a rescaling formally defined in Appendix B.1; it amounts to a multiplicative factor which depends on the number of batch passes). Interestingly, while the dynamics of the learning curves of Adam and AdamW often coincided for the first half of the training run, AdamW often led to lower training loss and test errors (see Figure 3 top left and top right, respectively). Importantly, the use of L_2 weight decay in Adam did not yield as good

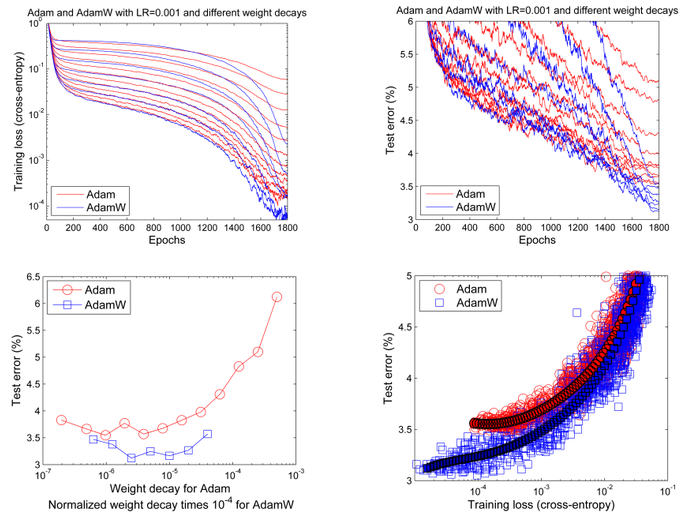


图3：在CIFAR-10数据集上，使用Adam优化器与AdamW算法分别训练的26 2x96d ResNet网络所得到的学习曲线（上排）及泛化性能结果（下排）。详情请参阅正文。附录中的补充图4展示了在ImageNet32x32数据集上得到的相同质量级别的结果。

采用L_2 正则化的随机梯度下降无法达到如此好的效果。

图2（左下角）展示了采用L_2 正则化的Adam优化器的实验结果。Adam优化器在最优超参数设置下的表现明显逊于随机梯度下降法的最优设置（参见图2左上角）。尽管这两种方法都使用了L_2 正则化，但Adam优化器并未从中获得任何好处：当L_2 正则化系数非零时，它所取得的最佳结果与未应用L_2 正则化时的最佳结果相当，也就是当 $\lambda = 0$时的情况。与原始的随机梯度下降法类似，超参数空间结构的形态表明这两个超参数之间存在耦合关系。

相比之下，图2（右下角）中采用解耦式权重衰减的Adam新变体（即AdamW）的实验结果表明，AdamW在很大程度上实现了权重衰减与学习率的解耦。在最优超参数设置下的表现显著优于采用L_2 正则化的Adam，其性能甚至可与随机梯度下降及SGDW算法相媲美。

总而言之，图2中的结果支持了我们的假设：权重衰减与学习率这两个超参数是可以解耦的，而这样做不仅简化了SGD中超参数调优的过程，还能提升Adam优化器的性能，使其能够与带有动量的SGD相媲美。

4.3 AdamW更优的泛化能力

尽管之前的实验表明AdamW的最优超参数搜索空间比Adam的更广阔、更深，接下来我们通过长达1800个训练轮数的更长时间运行来分析相关结果，以此比较AdamW与Adam的泛化能力。

我们将初始学习率设定为0.001，这一数值既是Adam的默认学习率，也在我们的实验中展现了相当不错的效果。图3展示了Adam的L_2 正则化在12种不同设置下的结果，以及AdamW的归一化权重衰减在7种不同设置下的结果（归一化权重衰减指的是附录B.1中正式定义的一种重新缩放方式；它实际上是一个乘法因子，其数值取决于批量迭代次数）。有趣的是，尽管在训练过程的前半段，Adam和AdamW的学习曲线走势往往相似，但AdamW通常能带来更低的训练损失和测试误差（分别见图3的左上角和右上角）。重要的是，虽然在Adam中使用L_2 权重衰减也取得了一定效果，但其效果并不如前者优异。

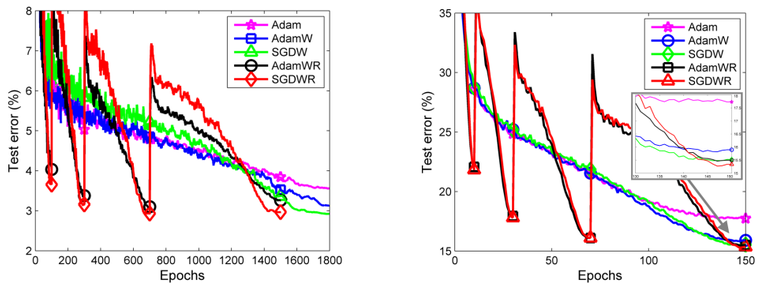


Figure 4: Top-1 test error on CIFAR-10 (left) and Top-5 test error on ImageNet32x32 (right). For a better resolution and with training loss curves, see SuppFigure 5 and SuppFigure 6 in the supplementary material.

results as decoupled weight decay in AdamW (see also Figure 3, bottom left). Next, we investigated whether AdamW's better results were only due to better convergence or due to better generalization. *The results in Figure 3 (bottom right) for the best settings of Adam and AdamW suggest that AdamW did not only yield better training loss but also yielded better generalization performance for similar training loss values*. The results on ImageNet32x32 (see SuppFigure 4 in the Appendix) yield the same conclusion of substantially improved generalization performance.

4.4 AdamWR with Warm Restarts for Better Anytime Performance

In order to improve the anytime performance of SGDW and AdamW we extended them with the warm restarts we introduced in Loshchilov & Hutter (2016), to obtain SGDWR and AdamWR, respectively (see Section B.2 in the Appendix). As Figure 4 shows, AdamWR greatly sped up AdamW on CIFAR-10 and ImageNet32x32, up to a factor of 10 (see the results at the first restart). For the default learning rate of 0.001, *AdamW achieved 15% relative improvement in test error compared to Adam both on CIFAR-10* (also see SuppFigure 5) *and ImageNet32x32* (also see SuppFigure 6).

AdamWR achieved the same improved results but with a much better anytime performance. These improvements closed most of the gap between Adam and SGDWR on CIFAR-10 and yielded comparable performance on ImageNet32x32.

4.5 Use of AdamW on other datasets and architectures

Several other research groups have already successfully applied AdamW in citable works. For example, Wang et al. (2018) used AdamW to train a novel architecture for face detection on the standard WIDER FACE dataset (Yang et al., 2016), obtaining almost 10x faster predictions than the previous state of the art algorithms while achieving comparable performance. Völker et al. (2018) employed AdamW with cosine annealing to train convolutional neural networks to classify and characterize error-related brain signals measured from intracranial electroencephalography (EEG) recordings. While their paper does not provide a comparison to Adam, they kindly provided us with a direct comparison of the two on their best-performing problem-specific network architecture Deep4Net and a variant of ResNet. AdamW with the same hyperparameter setting as Adam yielded higher test set accuracy on Deep4Net (73.68% versus 71.37%) and statistically significantly higher test set accuracy on ResNet (72.04% versus 61.34%). Radford et al. (2018) employed AdamW to train Transformer (Vaswani et al., 2017) architectures to obtain new state-of-the-art results on a wide range of benchmarks for natural language understanding. Zhang et al. (2018) compared L_2 regularization vs. weight decay for SGD, Adam and the Kronecker-Factored Approximate Curvature (K-FAC) optimizer (Martens & Grosse, 2015) on the CIFAR datasets with ResNet and VGG architectures, reporting that decoupled weight decay consistently outperformed L_2 regularization in cases where they differ.

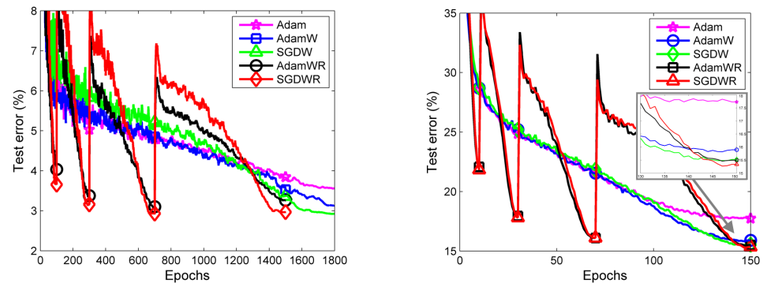


图4：CIFAR-10数据集的Top-1测试误差（左侧）以及ImageNet32x32数据集的Top-5测试误差（右侧）。如需更高分辨率的图像及训练损失曲线，请参阅补充材料中的SuppFigure 5和SuppFigure 6。

这些结果对应于AdamW中的解耦权重衰减策略（亦可参见图3的左下角）。接下来，我们研究了AdamW之所以能取得更好性能，究竟是源于更快的收敛速度，还是更强的泛化能力。图*3*右下角中*Adam*与*AdamW*在最佳参数设置下的实验结果表明，即使在训练损失相近的情况下，*AdamW*不仅能够实现更低的训练损失，还能展现出更好的泛化性能。在ImageNet32x32数据集上的测试结果（详见附录中的补充图4）也得出了同样的结论，即其泛化性能有了显著提升。

4.4 结合温重启的AdamWR算法，以实现更优异的任意时刻性能

为提升SGDW与AdamW的任意时刻性能，我们采用了在Loshchilov与Hutter（2016）中提出的温重启技术对它们进行扩展，从而分别得到了SGDWWR和AdamWR优化器（详见附录中的B.2节）。如图4所示，在CIFAR-10和ImageNet32x32数据集上，AdamWR使AdamW的运算速度提升了多达10倍（可查看首次重启时的测试结果）。在默认学习率为0.001的情况下，无论是在*CIFAR-10*（亦可参见补充图5）还是*ImageNet32x32*（亦可参见补充图6）数据集上，AdamW的测试误差相比Adam都实现了15%的相对降低。

*AdamWR*在获得与原有方法相同优化效果的同时，其任意时刻的运行性能也大幅提升。这些改进大幅缩小了*CIFAR-10*数据集上*Adam*优化器与*SGDWR*优化器之间的差距，同时在*ImageNet32x32*数据集上也展现出了相当出色的性能。

4.5 在其他数据集及架构上的AdamW应用

Several other research groups have already successfully applied AdamW in citable works. For example, Wang et al. (2018) used AdamW to train a novel architecture for face detection on the standard WIDER FACE dataset (Yang et al., 2016), obtaining almost 10x faster predictions than the previous state of the art algorithms while achieving comparable performance. Völker et al. (2018) employed AdamW with cosine annealing to train convolutional neural networks to classify and characterize error-related brain signals measured from intracranial electroencephalography (EEG) recordings. While their paper does not provide a comparison to Adam, they kindly provided us with a direct comparison of the two on their best-performing problem-specific network architecture Deep4Net and a variant of ResNet. AdamW with the same hyperparameter setting as Adam yielded higher test set accuracy on Deep4Net (73.68% versus 71.37%) and statistically significantly higher test set accuracy on ResNet (72.04% versus 61.34%). Radford et al. (2018) employed AdamW to train Transformer (Vaswani et al., 2017) architectures to obtain new state-of-the-art results on a wide range of benchmarks for natural language understanding. Zhang et al. (2018) compared L_2 regularization vs. weight decay for SGD, Adam and the Kronecker-Factored Approximate Curvature (K-FAC) optimizer (Martens & Grosse, 2015) on the CIFAR datasets with ResNet and VGG architectures, reporting that decoupled weight decay consistently outperformed L_2 regularization in cases where they differ.

5 CONCLUSION AND FUTURE WORK

Following suggestions that adaptive gradient methods such as Adam might lead to worse generalization than SGD with momentum (Wilson et al., 2017), we identified and exposed the inequivalence of L_2 regularization and weight decay for Adam. We empirically showed that our version of Adam with decoupled weight decay yields substantially better generalization performance than the common implementation of Adam with L_2 regularization. We also proposed to use warm restarts for Adam to improve its anytime performance.

Our results obtained on image classification datasets must be verified on a wider range of tasks, especially ones where the use of regularization is expected to be important. It would be interesting to integrate our findings on weight decay into other methods which attempt to improve Adam, e.g. normalized direction-preserving Adam (Zhang et al., 2017). While we focused our experimental analysis on Adam, we believe that similar results also hold for other adaptive gradient methods, such as AdaGrad (Duchi et al., 2011) and AMSGrad (Reddi et al., 2018).

6 ACKNOWLEDGMENTS

We thank Patryk Chrabaszcz for help with running experiments with ImageNet32x32; Matthias Feurer and Robin Schirrmeister for providing valuable feedback on this paper in several iterations; and Martin Völker, Robin Schirrmeister, and Tonio Ball for providing us with a comparison of AdamW and Adam on their EEG data. We also thank the following members of the deep learning community for implementing decoupled weight decay in various deep learning libraries:

- Jingwei Zhang, Lei Tai, Robin Schirrmeister, and Kashif Rasul for their implementations in PyTorch (see `https://github.com/pytorch/pytorch/pull/4429`)
- Phil Jund for his implementation in TensorFlow described at `https://www.tensorflow.org/api_docs/python/tf/contrib/opt/DecoupledWeightDecayExtension`
- Sylvain Gugger, Anand Saha, Jeremy Howard and other members of fast.ai for their implementation available at `https://github.com/sgugger/Adam-experiments`
- Guillaume Lambard for his implementation in Keras available at `https://github.com/GLambard/AdamW_Keras`
- Yagami Lin for his implementation in Caffe available at `https://github.com/Yagami123/Caffe-AdamW-AdamWR`

This work was supported by the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation programme under grant no. 716721, by the German Research Foundation (DFG) under the BrainLinksBrainTools Cluster of Excellence (grant number EXC 1086) and through grant no. INST 37/935-1 FUGG, and by the German state of Baden-Württemberg through bwHPC.

REFERENCES

Laurence Aitchison. A unified theory of adaptive stochastic gradient descent as Bayesian filtering. *arXiv:1507.02030*, 2018.

Patryk Chrabaszcz, Ilya Loshchilov, and Frank Hutter. A downsampled variant of ImageNet as an alternative to the CIFAR datasets. *arXiv:1707.08819*, 2017.

Ekin D Cubuk, Barret Zoph, Dandelion Mane, Vijay Vasudevan, and Quoc V Le. Autoaugment: Learning augmentation policies from data. *arXiv preprint arXiv:1805.09501*, 2018.

Laurent Dinh, Razvan Pascanu, Samy Bengio, and Yoshua Bengio. Sharp minima can generalize for deep nets. *arXiv:1703.04933*, 2017.

John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *The Journal of Machine Learning Research*, 12:2121–2159, 2011.

5 结论与未来工作

基于有研究指出诸如Adam优化器这类自适应梯度方法可能会比带有动量的随机梯度下降算法具有更差的泛化能力（Wilson等人，2017年）这一观点，我们揭示了针对Adam算法的L_2 正则化与权重衰减方法之间的不等价性。通过实验证明，采用解耦式权重衰减机制的Adam版本，其泛化性能明显优于传统使用L_2 正则化的Adam实现方式。此外，我们还提出了为Adam算法引入温重启技术，以此提升其在任意时刻下的运行性能。

我们在图像分类数据集上获得的结果还需在更广泛的任务中进行验证，尤其是那些预期会用到权重衰减技术的任务。将我们关于权重衰减的发现应用到那些旨在改进Adam优化器的其他方法中，例如归一化方向保持Adam算法（Zhang等人，2017年），将会非常有意义。尽管我们的实验分析主要针对Adam优化器展开，但我们认为类似的结果也适用于其他自适应梯度方法，比如AdaGrad（Duchi等人，2011年）和AMSGrad（Reddi等人，2018年）。

6 致谢

我们要感谢Patryk Chrabaszcz在使用ImageNet32x32数据集进行实验时给予的帮助；感谢Matthias Feurer和Robin Schirrmeister在论文的多次修改过程中提供的宝贵意见；同时也要感谢Martin V¨olker、Robin Schirrmeister以及Tonio Ball为我们提供了他们在EEG数据上对AdamW算法与Adam优化器进行对比的结果。此外，我们还要感谢深度学习领域的以下同仁们，他们分别在不同的深度学习库中实现了解耦式权重衰减功能。

感谢• Jingwei Zhang、Lei Tai、Robin Schirrmeister和Kashif Rasul在PyTorch中的实现工作（详见https://github.com/pytorch/pytorch/pull/4429）
• 感谢Phil Jund在https://www.tensorflow.org/api中实现的基于TensorFlow的解决方案_docs/python/tf/contrib/opt/ DecoupledWeightDecayExtension

• Sylvain Gugger、Anand Saha、Jeremy Howard以及fast.ai的其余成员，感谢他们实现了该算法，相关代码可见于https://github.com/sgugger/Adam-experiments
• 感谢Guillaume Lambard在Keras中实现的该算法，相关代码可见于https://github.com/GLambard/AdamW_Keras
• 感谢Yagami Lin实现了基于Caffe的相应代码，该代码可访问地址：https://github.com/Yagami123/Caffe-AdamW-AdamWR

本项工作得到了欧洲研究委员会（ERC）在欧盟“地平线2020”研究与创新计划下的资助，项目编号为716721；同时获得了德国研究基金会（DFG）在BrainLinksBrainTools卓越集群项目下的支持（项目编号EXC 1086），以及编号为INST 37/935-1 FUGG的资助；此外还得到了巴登-符腾堡州通过bwHPC项目提供的经费支持。

REFERENCES

LaurEnce Aitchison所提出的理论将自适应随机梯度下降统一视为一种贝叶斯滤波方法。*arXiv:1507.02030*，2018年。

Patryk Chrabaszcz、Ilya Loshchilov与Frank Hutter提出了基于ImageNet降采样版本的数据集，作为CIFAR数据集的替代方案。*arXiv:1707.08819*，2017年。

Ekin D Cubuk、巴雷特·佐夫、Dandelion Mane、维贾伊·瓦苏德万以及Quoc V Le共同提出了Autoaugment方法，该方法能够从数据中学习数据增强策略。*arXiv*预印本 *arXiv:1805.09501*，2018年。

Laurent Dinh、Razvan Pascanu、Samy Bengio以及Yoshua Bengio指出，深度神经网络中的尖锐极小值具备泛化能力。*arXiv:1703.04933*，2017年。

John Duchi、Elad Hazan与Yoram Singer。《用于在线学习与随机优化的自适应次梯度方法》。《机器学习研究杂志》，12:2121–2159，2011年。

Xavier Gastaldi. Shake-Shake regularization. *arXiv preprint arXiv:1705.07485*, 2017.

Stephen José Hanson and Lorien Y Pratt. Comparing biases for minimal network construction with back-propagation. In *Proceedings of the 1st International Conference on Neural Information Processing Systems*, pp. 177–185, 1988.

Gao Huang, Yixuan Li, Geoff Pleiss, Zhuang Liu, John E Hopcroft, and Kilian Q Weinberger. Snapshot ensembles: Train 1, get m for free. *arXiv:1704.00109*, 2017.

Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalization gap and sharp minima. *arXiv:1609.04836*, 2016.

Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv:1412.6980*, 2014.

Alex Krizhevsky. Learning multiple layers of features from tiny images. 2009.

Hao Li, Zheng Xu, Gavin Taylor, and Tom Goldstein. Visualizing the loss landscape of neural nets. *arXiv preprint arXiv:1712.09913*, 2017.

Ilya Loshchilov and Frank Hutter. SGDR: stochastic gradient descent with warm restarts. *arXiv:1608.03983*, 2016.

James Martens and Roger Grosse. Optimizing neural networks with kronecker-factored approximate curvature. In *International conference on machine learning*, pp. 2408–2417, 2015.

Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv:1511.06434*, 2015.

Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. *URL https://s3-us-west-2. amazonaws. com/openai-assets/research-covers/language-unsupervised/language_ understanding_paper. pdf*, 2018.

Sashank J. Reddi, Satyen Kale, and Sanjiv Kumar. On the convergence of adam and beyond. *International Conference on Learning Representations*, 2018.

Leslie N Smith. Cyclical learning rates for training neural networks. *arXiv:1506.01186v3*, 2016.

Tijmen Tieleman and Geoffrey Hinton. Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude. *COURSERA: Neural networks for machine learning*, 4(2):26–31, 2012.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, pp. 5998–6008, 2017.

Martin Völker, Jiří Hammer, Robin T Schirrmeister, Joos Behncke, Lukas DJ Fiederer, Andreas Schulze-Bonhage, Petr Marusič, Wolfram Burgard, and Tonio Ball. Intracranial error detection via deep learning. *arXiv preprint arXiv:1805.01667*, 2018.

Jianfeng Wang, Ye Yuan, Gang Yu, and Sun Jian. Sface: An efficient network for face detection in large scale variations. *arXiv preprint arXiv:1804.06559*, 2018.

Ashia C Wilson, Rebecca Roelofs, Mitchell Stern, Nathan Srebro, and Benjamin Recht. The marginal value of adaptive gradient methods in machine learning. *arXiv:1705.08292*, 2017.

Kelvin Xu, Jimmy Ba, Ryan Kiros, Kyunghyun Cho, Aaron Courville, Ruslan Salakhudinov, Rich Zemel, and Yoshua Bengio. Show, attend and tell: Neural image caption generation with visual attention. In *International Conference on Machine Learning*, pp. 2048–2057, 2015.

Shuo Yang, Ping Luo, Chen-Change Loy, and Xiaoou Tang. Wider face: A face detection benchmark. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 5525–5533, 2016.

Xavier Gastaldi. Shake-Shake regularization. *arXiv preprint arXiv:1705.07485*, 2017.

Stephen José Hanson and Lorien Y Pratt. Comparing biases for minimal network construction with back-propagation. In *Proceedings of the 1st International Conference on Neural Information Processing Systems*, pp. 177–185, 1988.

Gao Huang, Yixuan Li, Geoff Pleiss, Zhuang Liu, John E Hopcroft, and Kilian Q Weinberger. Snapshot ensembles: Train 1, get m for free. *arXiv:1704.00109*, 2017.

Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalization gap and sharp minima. *arXiv:1609.04836*, 2016.

Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv:1412.6980*, 2014.

Alex Krizhevsky. Learning multiple layers of features from tiny images. 2009.

Hao Li, Zheng Xu, Gavin Taylor, and Tom Goldstein. Visualizing the loss landscape of neural nets. *arXiv preprint arXiv:1712.09913*, 2017.

Ilya Loshchilov and Frank Hutter. SGDR: stochastic gradient descent with warm restarts. *arXiv:1608.03983*, 2016.

James Martens and Roger Grosse. Optimizing neural networks with kronecker-factored approximate curvature. In *International conference on machine learning*, pp. 2408–2417, 2015.

Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv:1511.06434*, 2015.

Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. *URL https://s3-us-west-2. amazonaws. com/openai-assets/research-covers/language-unsupervised/language_ understanding_paper. pdf*, 2018.

Sashank J. Reddi, Satyen Kale, and Sanjiv Kumar. On the convergence of adam and beyond. *International Conference on Learning Representations*, 2018.

Leslie N Smith. Cyclical learning rates for training neural networks. *arXiv:1506.01186v3*, 2016.

Tijmen Tieleman and Geoffrey Hinton. Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude. *COURSERA: Neural networks for machine learning*, 4(2):26–31, 2012.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, pp. 5998–6008, 2017.

Martin Völker, Jiří Hammer, Robin T Schirrmeister, Joos Behncke, Lukas DJ Fiederer, Andreas Schulze-Bonhage, Petr Marusič, Wolfram Burgard, and Tonio Ball. Intracranial error detection via deep learning. *arXiv preprint arXiv:1805.01667*, 2018.

Jianfeng Wang, Ye Yuan, Gang Yu, and Sun Jian. Sface: An efficient network for face detection in large scale variations. *arXiv preprint arXiv:1804.06559*, 2018.

Ashia C Wilson, Rebecca Roelofs, Mitchell Stern, Nathan Srebro, and Benjamin Recht. The marginal value of adaptive gradient methods in machine learning. *arXiv:1705.08292*, 2017.

Kelvin Xu, Jimmy Ba, Ryan Kiros, Kyunghyun Cho, Aaron Courville, Ruslan Salakhudinov, Rich Zemel, and Yoshua Bengio. Show, attend and tell: Neural image caption generation with visual attention. In *International Conference on Machine Learning*, pp. 2048–2057, 2015.

Shuo Yang, Ping Luo, Chen-Change Loy, and Xiaoou Tang. Wider face: A face detection benchmark. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 5525–5533, 2016.

Guodong Zhang, Chaoqi Wang, Bowen Xu, and Roger Grosse. Three mechanisms of weight decay regularization. *arXiv preprint arXiv:1810.12281*, 2018.

Zijun Zhang, Lin Ma, Zongpeng Li, and Chuan Wu. Normalized direction-preserving adam. *arXiv:1709.04546*, 2017.

Barret Zoph, Vijay Vasudevan, Jonathon Shlens, and Quoc V. Le. Learning transferable architectures for scalable image recognition. In *arXiv:1707.07012 [cs.CV]*, 2017.

张国东、王超奇、徐博文与罗杰·格罗斯共同提出了权重衰减正则化的三种机制。*arXiv*预印本 *arXiv:1810.12281*，2018年。

张志军、马琳、李宗鹏与吴川。归一化方向保持Adam算法。*arXiv:1709.04546*，2017年。

巴雷特·佐夫、维贾伊·瓦苏德万、乔纳森·施伦斯与科克·V·勒。面向可扩展图像识别任务的迁移式架构学习。载于*arXiv:1707.07012 [cs.CV]*, 2017。

Appendix

A Formal Analysis of Weight Decay vs L_2 Regularization

Proof of Proposition 1
The proof for this well-known fact is straight-forward. SGD without weight decay has the following iterates on $f_t^{\text{reg}}(\boldsymbol{\theta}) = f_t(\boldsymbol{\theta}) + \frac{\lambda'}{2}\left\|\boldsymbol{\theta}\right\|_2^2$:

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \alpha\nabla f_t^{\text{reg}}(\boldsymbol{\theta}_t) = \boldsymbol{\theta}_t - \alpha\nabla f_t(\boldsymbol{\theta}_t) - \alpha\lambda'\boldsymbol{\theta}_t. \tag{5}$$

SGD with weight decay has the following iterates on $f_t(\boldsymbol{\theta})$:

$$\boldsymbol{\theta}_{t+1} \leftarrow (1-\lambda)\boldsymbol{\theta}_t - \alpha\nabla f_t(\boldsymbol{\theta}_t). \tag{6}$$

These iterates are identical since $\lambda' = \frac{\lambda}{\alpha}$. □

Proof of Proposition 2
Similarly to the proof of Proposition 1, the iterates of O without weight decay on $f_t^{\text{reg}}(\boldsymbol{\theta}) = f_t(\boldsymbol{\theta}) + \frac{1}{2}\lambda'\left\|\boldsymbol{\theta}\right\|_2^2$ and O with weight decay λ on f_t are, respectively:

$$\begin{aligned}\boldsymbol{\theta}_{t+1} &\leftarrow \boldsymbol{\theta}_t - \alpha\lambda'\mathbf{M}_t\boldsymbol{\theta}_t - \alpha\mathbf{M}_t\nabla f_t(\boldsymbol{\theta}_t). &(7)\\ \boldsymbol{\theta}_{t+1} &\leftarrow (1-\lambda)\boldsymbol{\theta}_t - \alpha\mathbf{M}_t\nabla f_t(\boldsymbol{\theta}_t). &(8)\end{aligned}$$

The equality of these iterates for all $\boldsymbol{\theta}_t$ would imply $\lambda\boldsymbol{\theta}_t = \alpha\lambda'\mathbf{M}_t\boldsymbol{\theta}_t$. This can only hold for all $\boldsymbol{\theta}_t$ if $\mathbf{M}_t = k\mathbf{I}$, with $k \in \mathbb{R}$, which is not the case for O. Therefore, no L_2 regularizer $\lambda'\left\|\boldsymbol{\theta}\right\|_2^2$ exists that makes the iterates equivalent. □

Proof of Proposition 3
O without weight decay has the following iterates on $f_t^{\text{sreg}}(\boldsymbol{\theta}) = f_t(\boldsymbol{\theta}) + \frac{\lambda'}{2}\left\|\boldsymbol{\theta}\odot\sqrt{\boldsymbol{s}}\right\|_2^2$:

$$\begin{aligned}\boldsymbol{\theta}_{t+1} &\leftarrow \boldsymbol{\theta}_t - \alpha\nabla f_t^{\text{sreg}}(\boldsymbol{\theta}_t)/\boldsymbol{s} &(9)\\ &= \boldsymbol{\theta}_t - \alpha\nabla f_t(\boldsymbol{\theta}_t)/\boldsymbol{s} - \alpha\lambda'\boldsymbol{\theta}_t\odot\boldsymbol{s}/\boldsymbol{s} &(10)\\ &= \boldsymbol{\theta}_t - \alpha\nabla f_t(\boldsymbol{\theta}_t)/\boldsymbol{s} - \alpha\lambda'\boldsymbol{\theta}_t, &(11)\end{aligned}$$

where the division by $\boldsymbol{s}$ is element-wise. O with weight decay has the following iterates on $f_t(\boldsymbol{\theta})$:

$$\begin{aligned}\boldsymbol{\theta}_{t+1} &\leftarrow (1-\lambda)\boldsymbol{\theta}_t - \alpha\nabla f(\boldsymbol{\theta}_t)/\boldsymbol{s} &(12)\\ &= \boldsymbol{\theta}_t - \alpha\nabla f(\boldsymbol{\theta}_t)/\boldsymbol{s} - \lambda\boldsymbol{\theta}_t, &(13)\end{aligned}$$

These iterates are identical since $\lambda' = \frac{\lambda}{\alpha}$. □

B Additional Practical Improvements of Adam

Having discussed decoupled weight decay for improving Adam's generalization, in this section we introduce two additional components to improve Adam's performance in practice.

B.1 Normalized Weight Decay

Our preliminary experiments showed that different weight decay factors are optimal for different computational budgets (defined in terms of the number of batch passes). Relatedly, Li et al. (2017) demonstrated that a smaller batch size (for the same total number of epochs) leads to the shrinking effect of weight decay being more pronounced. Here, we propose to reduce this dependence by normalizing the values of weight decay. Specifically, we replace the hyperparameter λ by a new (more robust) normalized weight decay hyperparameter λ_{norm}, and use this to set λ as $\lambda = \lambda_{norm}\sqrt{\frac{b}{BT}}$, where b is the batch size, B is the total number of training points and T is the total number of epochs.[2] Thus, λ_{norm} can be interpreted as the weight decay used if only one batch pass is allowed. We emphasize that our choice of normalization is merely one possibility informed by few experiments; a more lasting conclusion we draw is that using *some* normalization can substantially improve results.

[2] In the context of our AdamWR variant discussed in Section B.2, T is the total number of epochs in the current restart.

附录

权重衰减VSL_2 正则化的形式化分析

命题1的证明
这一广为人知的事实其证明过程十分直接。在没有权重衰减的随机梯度下降算法中， $f_t^{\text{reg}}(\boldsymbol{\theta}) = f_t(\boldsymbol{\theta}) + \frac{\lambda'}{2}\left\|\boldsymbol{\theta}\right\|_2^2$ 上的迭代过程如下：

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \alpha\nabla f_t^{\text{reg}}(\boldsymbol{\theta}_t) = \boldsymbol{\theta}_t - \alpha\nabla f_t(\boldsymbol{\theta}_t) - \alpha\lambda'\boldsymbol{\theta}_t. \tag{5}$$

应用了权重衰减的随机梯度下降算法在 $f_t(\boldsymbol{\theta})$上的迭代过程如下：

$$\boldsymbol{\theta}_{t+1} \leftarrow (1-\lambda)\boldsymbol{\theta}_t - \alpha\nabla f_t(\boldsymbol{\theta}_t). \tag{6}$$

由于 $\lambda' = \frac{\lambda}{\alpha}$ 的原因，这两种迭代过程实际上是完全相同的。 □

命题2的证明 Si
milar命题1插出过程在 $f_t^{\text{reg}}(\boldsymbol{\theta}) = f_t(\boldsymbol{\theta}) + \frac{1}{2}\lambda'\left\|\boldsymbol{\theta}\right\|_2^2$ 上不应用权重衰减时， O 的迭代序列与在 f_t 上应用权重衰减 λ 时的迭代序列分别为：

$$\begin{aligned}\boldsymbol{\theta}_{t+1} &\leftarrow \boldsymbol{\theta}_t - \alpha\lambda'\mathbf{M}_t\boldsymbol{\theta}_t - \alpha\mathbf{M}_t\nabla f_t(\boldsymbol{\theta}_t). &(7)\\ \boldsymbol{\theta}_{t+1} &\leftarrow (1-\lambda)\boldsymbol{\theta}_t - \alpha\mathbf{M}_t\nabla f_t(\boldsymbol{\theta}_t). &(8)\end{aligned}$$

若所有 $\boldsymbol{\theta}_t$ 对应的这些迭代序列都相等，那么就会得出 $\lambda\boldsymbol{\theta}_t = \alpha\lambda'\mathbf{M}_t\boldsymbol{\theta}_t$这一结论。但只有当$\mathbf{M}_t = k$ I</style}且满足 $k \in \mathbb{R}$的条件时，这一关系才成立，而 O并不符合该条件。因此，不存在能够使这两种迭代序列等价的L_2 正则化项 $\lambda'\left\|\boldsymbol{\theta}\right\|_2^2$。 □

命题3的证明
O 在没有权重衰减的情况下， $f_t^{\text{sreg}}(\boldsymbol{\theta}) = f_t(\boldsymbol{\theta}) + \frac{\lambda'}{2}\left\|\boldsymbol{\theta}\odot\sqrt{\boldsymbol{s}}\right\|_2^2$的迭代过程如下：

$$\begin{aligned}\boldsymbol{\theta}_{t+1} &\leftarrow \boldsymbol{\theta}_t - \alpha\nabla f_t^{\text{sreg}}(\boldsymbol{\theta}_t)/\boldsymbol{s} &(9)\\ &= \boldsymbol{\theta}_t - \alpha\nabla f_t(\boldsymbol{\theta}_t)/\boldsymbol{s} - \alpha\lambda'\boldsymbol{\theta}_t\odot\boldsymbol{s}/\boldsymbol{s} &(10)\\ &= \boldsymbol{\theta}_t - \alpha\nabla f_t(\boldsymbol{\theta}_t)/\boldsymbol{s} - \alpha\lambda'\boldsymbol{\theta}_t, &(11)\end{aligned}$$

其中，$\boldsymbol{s}$ 所对应的除法操作是按元素逐项进行的。而在应用了权重衰减的 O 中， $f_t(\boldsymbol{\theta})$的迭代过程则为：

$$\begin{aligned}\boldsymbol{\theta}_{t+1} &\leftarrow (1-\lambda)\boldsymbol{\theta}_t - \alpha\nabla f(\boldsymbol{\theta}_t)/\boldsymbol{s} &(12)\\ &= \boldsymbol{\theta}_t - \alpha\nabla f(\boldsymbol{\theta}_t)/\boldsymbol{s} - \lambda\boldsymbol{\theta}_t, &(13)\end{aligned}$$

由于$\lambda' = \frac{\lambda}{\alpha}$的原因，这两种情况下的迭代结果是完全一致的。 □

B. 额外的Adam优化器实用改进措施

在探讨了通过解耦式权重衰减来提升Adam优化器泛化能力的做法之后，本节将进一步介绍两种额外的机制，用以在实际应用中进一步改善Adam的性能。

B.1 归一化权重衰减

我们的初步实验表明，不同的权重衰减系数适用于不同的计算资源限制（以批量迭代次数来界定）。与此相关的是，Li等人（2017）的研究指出，在总训练轮数相同的情况下，较小的批量大小会使得权重衰减的收缩效应更加显著。在此，我们提出通过对权重衰减的值进行归一化处理来降低这种依赖性。具体而言，我们用一个新的（更优的）参数来替代超参数 λ。

强大的归一化权重衰减超参数 λ_{norm}，并以此将 λ 设置为 $\lambda = \lambda_{norm}\sqrt{\frac{b}{BT}}$，其中 b 表示批量大小， B 表示训练样本的总数量， T 则表示训练轮数的总数。[2] 因此， λ_{norm} 可被理解为在仅允许进行一次批量处理时的权重衰减值。需要强调的是，我们所选择的归一化方法只是基于少量实验得出的一个可能方案；我们更确信的结论是，采用某种归一化方法能够显著提升算法性能。

[2] 在B.2节中讨论的AdamWR变体背景下， T 指的是当前重新开始训练时的总训练轮数。

B.2 ADAM WITH COSINE ANNEALING AND WARM RESTARTS

We now apply cosine annealing and warm restarts to Adam, following our recent work (Loshchilov & Hutter, 2016). There, we proposed Stochastic Gradient Descent with Warm Restarts (SGDR) to improve the anytime performance of SGD by quickly cooling down the learning rate according to a cosine schedule and periodically increasing it. SGDR has been successfully adopted to lead to new state-of-the-art results for popular image classification benchmarks (Huang et al., 2017; Gastaldi, 2017; Zoph et al., 2017), and we therefore already tried extending it to Adam shortly after proposing it. However, while our initial version of Adam with warm restarts had better anytime performance than Adam, it was not competitive with SGD with warm restarts, precisely because L_2 regularization was not working as well as in SGD. Now, having fixed this issue by means of the original weight decay regularization (Section 2) and also having introduced normalized weight decay (Section B.1), our original work on cosine annealing and warm restarts directly carries over to Adam.

In the interest of keeping the presentation self-contained, we briefly describe how SGDR schedules the change of the effective learning rate in order to accelerate the training of DNNs. Here, we decouple the initial learning rate α and its multiplier η_t used to obtain the actual learning rate at iteration t (see, e.g., line 8 in Algorithm 1). In SGDR, we simulate a new warm-started run/restart of SGD once T_i epochs are performed, where i is the index of the run. Importantly, the restarts are not performed from scratch but emulated by increasing η_t while the old value of $\boldsymbol{\theta}_t$ is used as an initial solution. The amount by which η_t is increased controls to which extent the previously acquired information (e.g., momentum) is used. Within the i-th run, the value of η_t decays according to a cosine annealing (Loshchilov & Hutter, 2016) learning rate for each batch as follows:

$$\eta_t = \eta_{min}^{(i)} + 0.5(\eta_{max}^{(i)} - \eta_{min}^{(i)})(1 + \cos(\pi T_{cur}/T_i)), \tag{14}$$

where $\eta_{min}^{(i)}$ and $\eta_{max}^{(i)}$ are ranges for the multiplier and T_{cur} accounts for how many epochs have been performed since the last restart. T_{cur} is updated at each batch iteration t and is thus not constrained to integer values. Adjusting (e.g., decreasing) $\eta_{min}^{(i)}$ and $\eta_{max}^{(i)}$ at every i-th restart (see also Smith (2016)) could potentially improve performance, but we do not consider that option here because it would involve additional hyperparameters. For $\eta_{max}^{(i)} = 1$ and $\eta_{min}^{(i)} = 0$, one can simplify Eq. (14) to

$$\eta_t = 0.5 + 0.5\cos(\pi T_{cur}/T_i). \tag{15}$$

In order to achieve good anytime performance, one can start with an initially small T_i (e.g., from 1% to 10% of the expected total budget) and multiply it by a factor of T_{mult} (e.g., $T_{mult} = 2$) at every restart. The $(i+1)$-th restart is triggered when $T_{cur} = T_i$ by setting T_{cur} to 0. An example setting of the schedule multiplier is given in C.

Our proposed **AdamWR** algorithm represents AdamW (see Algorithm 2) with η_t following Eq. (15) and λ computed at each iteration using normalized weight decay described in Section B.1. We note that normalized weight decay allowed us to use a constant parameter setting across short and long runs performed within AdamWR and SGDWR (SGDW with warm restarts).

C AN EXAMPLE SETTING OF THE SCHEDULE MULTIPLIER

An example schedule of the schedule multiplier η_t is given in SuppFigure 1 for $T_{i=0} = 100$ and $T_{mult} = 2$. After the initial 100 epochs the learning rate will reach 0 because $\eta_{t=100} = 0$. Then, since $T_{cur} = T_{i=0}$, we restart by resetting $T_{cur} = 0$, causing the multiplier η_t to be reset to 1 due to Eq. (15). This multiplier will then decrease again from 1 to 0, but now over the course of 200 epochs because $T_{i=1} = T_{i=0}T_{mult} = 200$. Solutions obtained right before the restarts, when $\eta_t = 0$ (e.g., at epoch indexes 100, 300, 700 and 1500 as shown in SuppFigure 1) are recommended by the optimizer as the solutions, with more recent solutions prioritized.

D ADDITIONAL RESULTS

We investigated whether the use of much longer runs (1800 epochs) of "standard Adam" (Adam with L_2 regularization and a fixed learning rate) makes the use of cosine annealing unnecessary.

B.2 应用余弦退火与温重启的Adam优化器

We now apply cosine annealing and warm restarts to Adam, following our recent work (Loshchilov & Hutter, 2016). There, we proposed Stochastic Gradient Descent with Warm Restarts (SGDR) to improve the anytime performance of SGD by quickly cooling down the learning rate according to a cosine schedule and periodically increasing it. SGDR has been successfully adopted to lead to new state-of-the-art results for popular image classification benchmarks (Huang et al., 2017; Gastaldi, 2017; Zoph et al., 2017), and we therefore already tried extending it to Adam shortly after proposing it. However, while our initial version of Adam with warm restarts had better anytime performance than Adam, it was not competitive with SGD with warm restarts, precisely because L_2 regularization was not working as well as in SGD. Now, having fixed this issue by means of the original weight decay regularization (Section 2) and also having introduced normalized weight decay (Section B.1), our original work on cosine annealing and warm restarts directly carries over to Adam.

为保证演示内容的完整性，我们简要介绍SGDR是如何安排有效学习率的变动，从而加速深度神经网络的训练的。在此，我们将初始学习率 α 及其用于在迭代 t 时计算实际学习率的乘数 η_t 分开处理（参见例如算法1中的第8行）。在SGDR中，一旦完成了 T_i 个训练轮数，就会模拟一次基于随机梯度下降的新热启动/重启过程，其中 i 代表该次运行的序号。重要的是，这些重启并非从零开始，而是通过不断增大 η_t 来模拟，同时仍以 $\boldsymbol{\theta}_t$ 的旧值作为初始解。 η_t 的增大幅度决定了先前获取的信息（如动量）会被利用到何种程度。在第 i次运行期间，每个批次的学习率会按照余弦退火（Loshchilov与Hutter，2016）规律逐渐衰减， η_t 的值也随之变化。

$$\eta_t = \eta_{min}^{(i)} + 0.5(\eta_{max}^{(i)} - \eta_{min}^{(i)})(1 + \cos(\pi T_{cur}/T_i)), \tag{14}$$

其中 $\eta_{min}^{(i)}$ 和 $\eta_{max}^{(i)}$ 分别是乘数的取值范围，而 T_{cur} 用于表示自上次重启以来已经完成了多少个训练轮数。 T_{cur} 会在每次 t 的批量迭代中更新，因此不必限制其为整数值。在每隔 i 次重启时调整（例如降低） $\eta_{min}^{(i)}$ 和 $\eta_{max}^{(i)}$ 的值（亦可参考Smith (2016)的研究）有望提升性能，但鉴于这会引入额外的超参数，我们在此并未考虑该方案。至于 $\eta_{max}^{(i)} = 1$和 $\eta_{min}^{(i)} = 0$，可以通过简化方程(14)来处理。

$$\eta_t = 0.5 + 0.5\cos(\pi T_{cur}/T_i). \tag{15}$$

为确保在任何时间点都能获得良好的性能，可以先设定一个初始较小的 T_i 值（例如预期总预算的1%到10%），然后在每次重启时将其乘以 T_{mult} 这个系数（例如 $T_{mult} = 2$）。当 $T_{cur} = T_i$ 时，通过将 T_{cur} 设为0来触发第$(i+1)$次重启。C部分给出了调度乘数的一个示例设置。

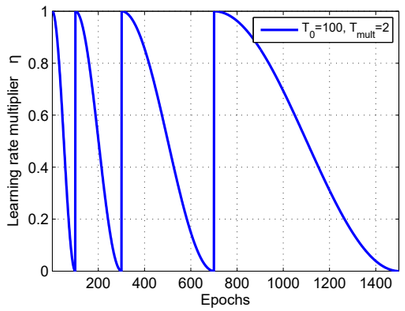
我们提出的**AdamWR** 算法是在AdamW（见算法2）的基础上，根据公式(15)中的 η_t 进行改进，并结合B.1节中所述的归一化权重衰减，在每次迭代时计算出 λ。需要指出的是，归一化权重衰减使得我们在AdamWR以及采用温重启机制的SGDW和SGDWR中进行短期与长期实验时，都能使用恒定的参数设置。

C：调度乘数的示例设置

补充图1为 $T_{i=0} = 100$ 和$T_{mult} = 2$给出了调度乘数 η_t 的一个示例时间表。在最初的100个训练轮数之后，由于 $\eta_{t=100} = 0$的原因，学习率将降至0。随后，基于 $T_{cur} = T_{i=0}$的设定，我们会通过重置 $T_{cur} = 0$来重新开始训练，根据公式(15)，这会导致乘数 η_t 被重置为1。之后该乘数又会从1逐渐下降至0，但这次是在200个训练轮数的过程中，原因在于 $T_{i=1} = T_{i=0}T_{mult} = 200$。优化器会优先选择在重新开始前获得的解，即当 $\eta_t = 0$时得到的解（如补充图1所示的训练轮数100、300、700和1500处的解），并且越新的解优先级越高。

D 其他结果

我们研究了使用运行时间更长的‘标准Adam算法’（即带有L_2 权重衰减机制且学习率固定的Adam优化器）——具体为1800个训练轮数——是否会让余弦退火变得不再必要。



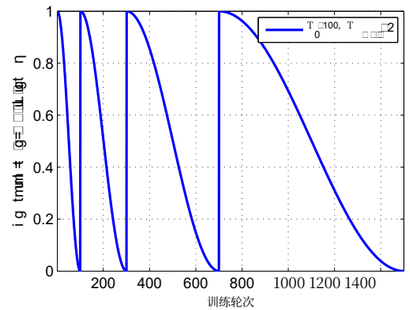
SuppFigure 1: An example schedule of the learning rate multiplier as a function of epoch index. The first run is scheduled to converge at epoch $T_{i=0} = 100$, then the budget for the next run is doubled as $T_{i=1} = T_{i=0}T_{mult} = 200$, etc.

SuppFigure 2 shows the results of standard Adam for a 4 by 4 logarithmic grid of hyperparameter settings (the coarseness of the grid is due to the high computational expense of runs for 1800 epochs). Even after taking the low resolution of the grid into account, the results appear to be at best comparable to the ones obtained with AdamW with 18 times less epochs and a smaller network (see SuppFigure 3, top row, middle). These results are not very surprising given Figure 1 in the main paper (which demonstrates both the improvements possible by using some learning rate schedule, such as cosine annealing, and the effectiveness of decoupled weight decay).

Our experimental results with Adam and SGD suggest that the total runtime in terms of the number of epochs affect the basin of optimal hyperparameters (see SuppFigure 3). More specifically, the greater the total number of epochs the smaller the values of the weight decay should be. SuppFigure 4 shows that our remedy for this problem, the normalized weight decay defined in Eq. (15), simplifies hyperparameter selection because the optimal values observed for short runs are similar to the ones for much longer runs. We used our initial experiments on CIFAR-10 to suggest the square root normalization we proposed in Eq. (15) and double-checked that this is not a coincidence on the ImageNet32x32 dataset (Chrabaszcz et al., 2017), a downsampled version of the original ImageNet dataset with 1.2 million 32×32 pixels images, where an epoch is 24 times longer than on CIFAR-10. This experiment also supported the square root scaling: the best values of the normalized weight decay observed on CIFAR-10 represented nearly optimal values for ImageNet32x32 (see SuppFigure 3). In contrast, had we used the same raw weight decay values λ for ImageNet32x32 as for CIFAR-10 and for the same number of epochs, *without the proposed normalization, λ would have been roughly 5 times too large for ImageNet32x32, leading to much worse performance*. The optimal normalized weight decay values were also very similar (e.g., $\lambda_{norm} = 0.025$ and $\lambda_{norm} = 0.05$) across SGDW and AdamW. These results clearly show that normalizing weight decay can substantially improve performance; while square root scaling performed very well in our experiments we emphasize that these experiments were not very comprehensive and that even better scaling rules are likely to exist.

SuppFigure 4 is the equivalent of Figure 3 in the main paper, but for ImageNet32x32 instead of for CIFAR-10. The qualitative results are identical: weight decay leads to better training loss (cross-entropy) than L_2 regularization, and to an even greater improvement of test error.

SuppFigure 5 and SuppFigure 6 are the equivalents of Figure 4 in the main paper but supplemented with training loss curves in its bottom row. The results show that Adam and its variants with decoupled weight decay converge faster (in terms of training loss) on CIFAR-10 than the corresponding SGD variants (the difference for ImageNet32x32 is small). As is discussed in the main paper, when the same values of training loss are considered, AdamW demonstrates better values of test error than Adam. Interestingly, SuppFigure 5 and SuppFigure 6 show that the restart variants AdamWR and SGDWR also demonstrate better generalization than AdamW and SGDW, respectively.



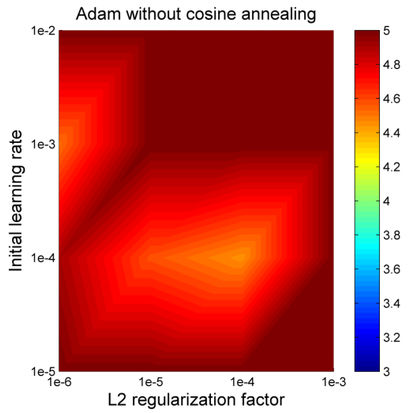
补充图1：学习率乘数随训练轮数变化的示例时间表。首次运行计划在训练轮数 $T_{i=0} = 100$ 时收敛，之后下次运行的预算会翻倍至 $T_{i=1} = T_{i=0}T_{mult} = 200$，以此类推。

补充图2展示了在4×4的对数尺度超参数网格上使用标准Adam算法所得的结果（由于1800个训练轮数的计算成本极高，因此网格的分辨率相对较低）。即便考虑到这一较低的分辨率，该算法的优化效果最多也只能与使用AdamW算法、训练轮数仅为其1/18且网络规模更小的方案相媲美（详见补充图3的最上面一行中间部分）。结合主论文中的图1——该图既展示了采用诸如余弦退火之类的学习率调度策略所能带来的改进，也体现了解耦式权重衰减的有效性——这些结果其实并不令人意外。

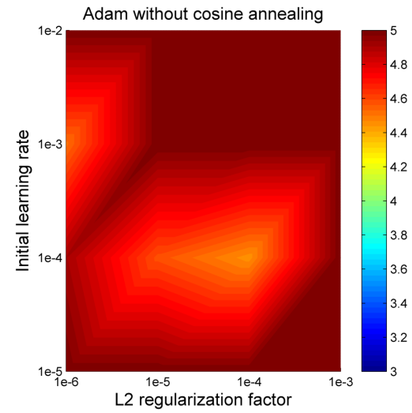
Our experimental results with Adam and SGD suggest that the total runtime in terms of the number of epochs affect the basin of optimal hyperparameters (see SuppFigure 3). More specifically, the greater the total number of epochs the smaller the values of the weight decay should be. SuppFigure 4 shows that our remedy for this problem, the normalized weight decay defined in Eq. (15), simplifies hyperparameter selection because the optimal values observed for short runs are similar to the ones for much longer runs. We used our initial experiments on CIFAR-10 to suggest the square root normalization we proposed in Eq. (15) and double-checked that this is not a coincidence on the ImageNet32x32 dataset (Chrabaszcz et al., 2017), a downsampled version of the original ImageNet dataset with 1.2 million 32×32 pixels images, where an epoch is 24 times longer than on CIFAR-10. This experiment also supported the square root scaling: the best values of the normalized weight decay observed on CIFAR-10 represented nearly optimal values for ImageNet32x32 (see SuppFigure 3). In contrast, had we used the same raw weight decay values λ for ImageNet32x32 as for CIFAR-10 and for the same number of epochs, *without the proposed normalization, λ would have been roughly 5 times too large for ImageNet32x32, leading to much worse performance*. The optimal normalized weight decay values were also very similar (e.g., $\lambda_{norm} = 0.025$ and $\lambda_{norm} = 0.05$) across SGDW and AdamW. These results clearly show that normalizing weight decay can substantially improve performance; while square root scaling performed very well in our experiments we emphasize that these experiments were not very comprehensive and that even better scaling rules are likely to exist.

补充图4对应于主论文中的图3，只不过所使用的数据集是从CIFAR-10换成了ImageNet32x32。两者的定性结果完全一致：与L_2正则化相比，权重衰减能够使训练损失（交叉熵损失）更低，同时还能进一步降低测试误差。

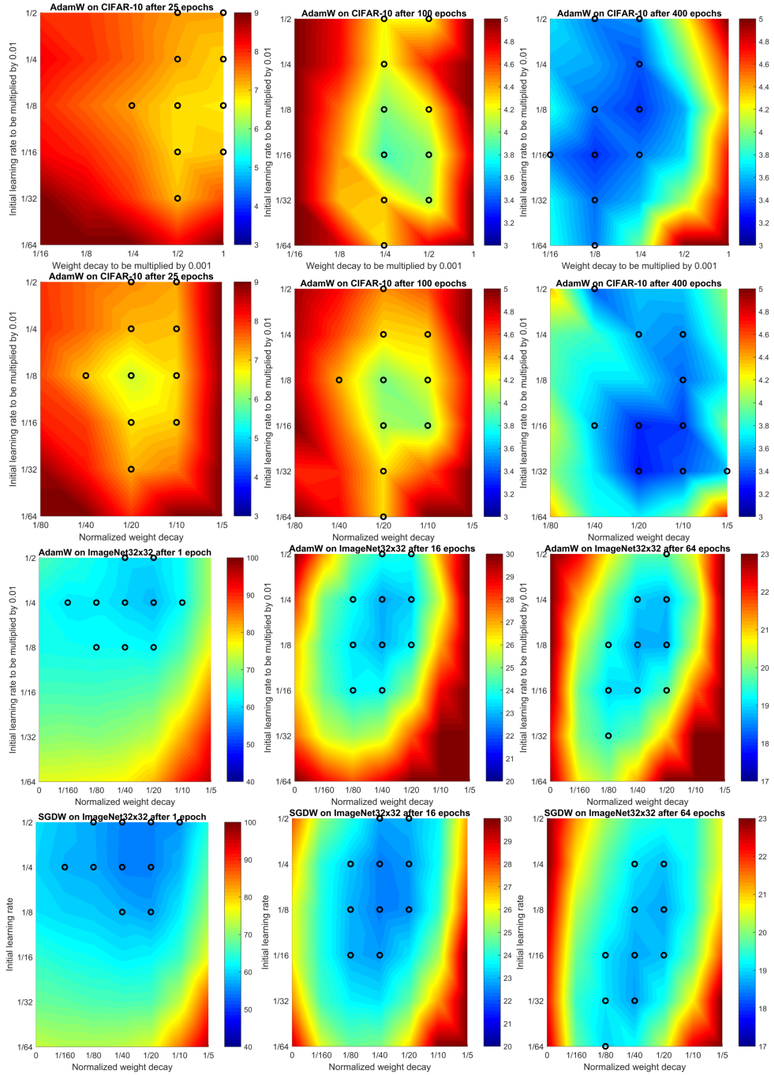
补充图5和补充图6同样对应于主论文中的图4，不同之处在于在底部添加了训练损失曲线。实验结果表明，在CIFAR-10数据集上，Adam及其采用解耦权重衰减的变体在训练损失下降的速度上要快于相应的随机梯度下降变体（而在ImageNet32x32数据集上两者差距较小）。正如主论文中所分析的，当训练损失数值相同时，AdamW的测试误差表现优于Adam。有趣的是，补充图5和补充图6还显示，采用重启机制的AdamWR和SGDWR变体在泛化能力方面也分别优于AdamW和SGDW。



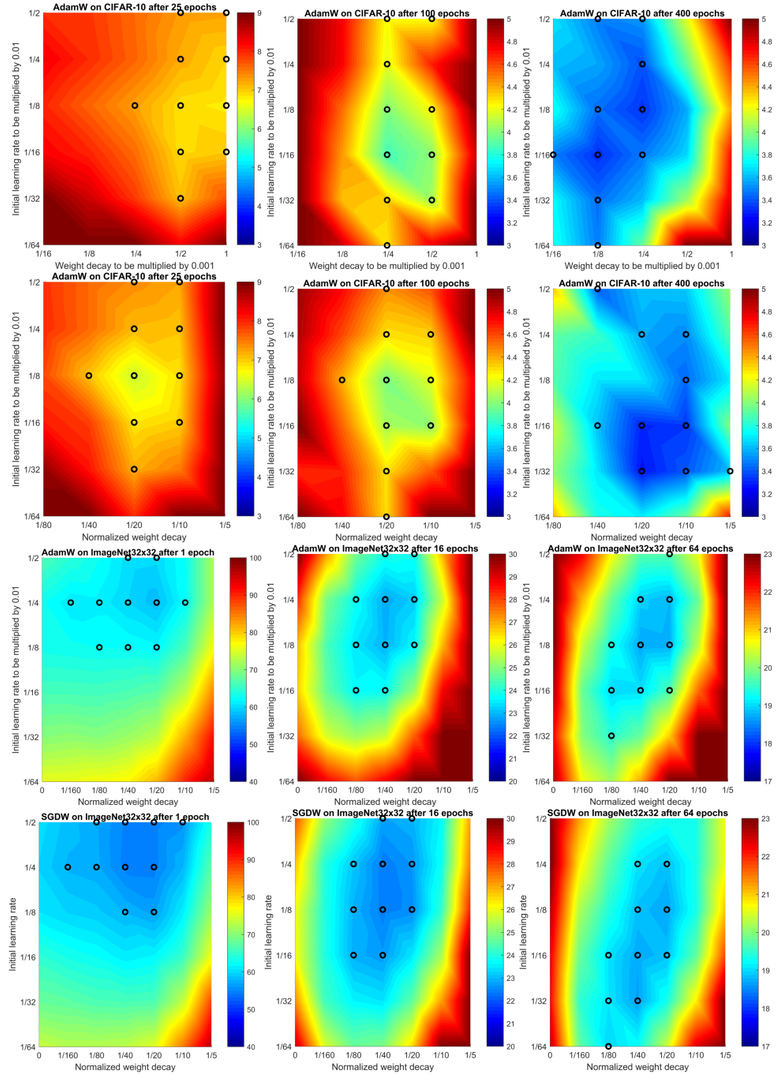
SuppFigure 2: Performance of "standard Adam": Adam with L_2 regularization and a fixed learning rate. We show the final test error of a 26 2x96d ResNet on CIFAR-10 after 1800 epochs of the original Adam for different settings of learning rate and weight decay used for L_2 regularization.



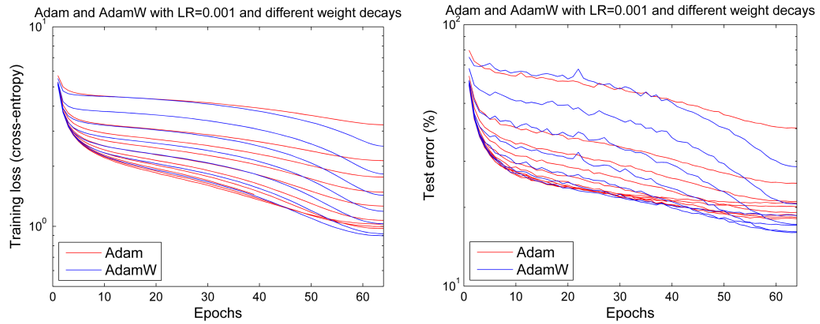
补充图2：‘标准Adam’的性能表现——即采用L_2正则化且学习率固定的Adam优化器。图中展示了在原始Adam优化器下运行1800个训练轮数后，针对不同的学习率及用于L_2正则化的权重衰减参数设置，26层2x96维度的ResNet网络在CIFAR-10数据集上的最终测试误差。

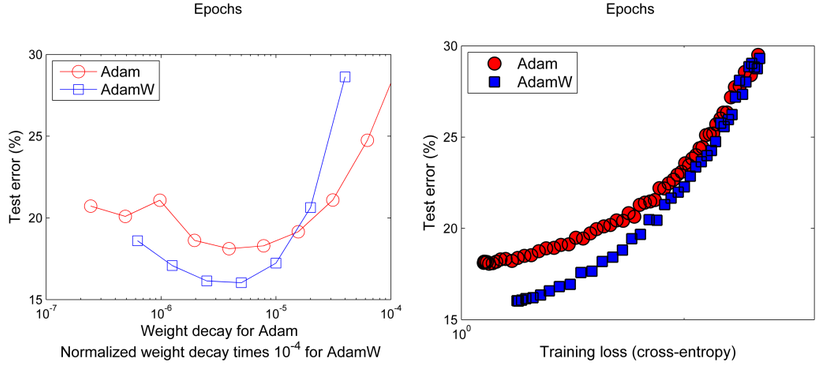


SuppFigure 3: Effect of normalized weight decay. We show the final test Top-1 error on CIFAR-10 (first two rows for AdamW without and with normalized weight decay) and Top-5 error on ImageNet32x32 (last two rows for AdamW and SGDW, both with normalized weight decay) of a 26 2x64d ResNet after different numbers of epochs (see columns). While the optimal settings of the raw weight decay change significantly for different runtime budgets (see the first row), the values of the normalized weight decay remain very similar for different budgets (see the second row) and different datasets (here, CIFAR-10 and ImageNet32x32), and even across AdamW and SGDW.

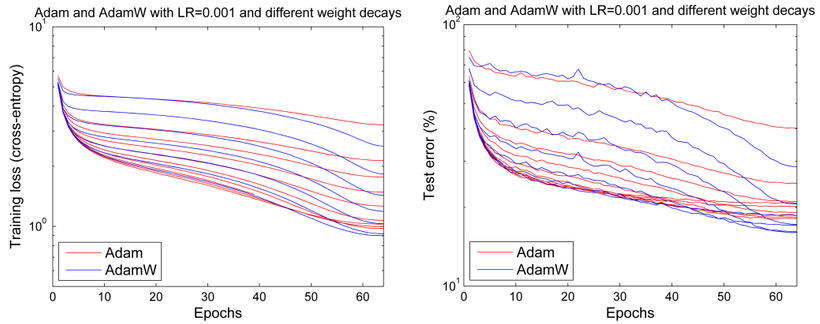


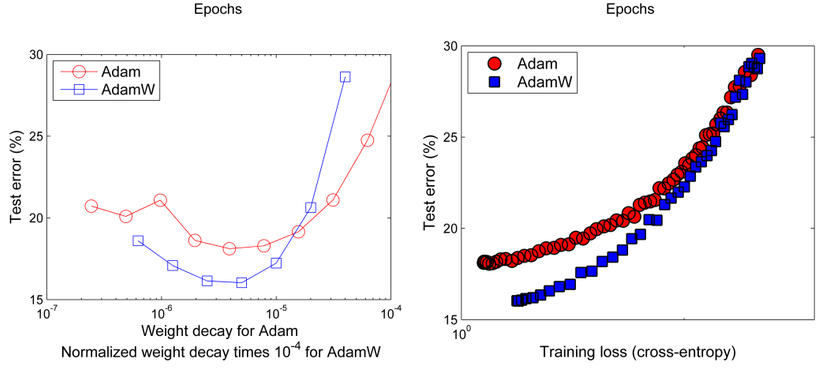
附图3：归一化权重衰减的效果。图中展示了经过不同训练轮数后、基于26层2x64维ResNet网络的CIFAR-10数据集的最终测试Top-1误差（前两行分别为未应用归一化权重衰减和应用了归一化权重衰减的AdamW算法结果），以及ImageNet32x32数据集的Top-5误差（最后两行为分别应用了归一化权重衰减的AdamW算法和SGDW算法的结果）。如各列所示，虽然不同的时间预算下原始权重衰减的最优参数会有显著差异（见第一行），但归一化权重衰减的数值在不同预算、不同数据集（此处为CIFAR-10和ImageNet32x32），甚至在不同算法（AdamW与SGDW）之间，依然保持高度一致。





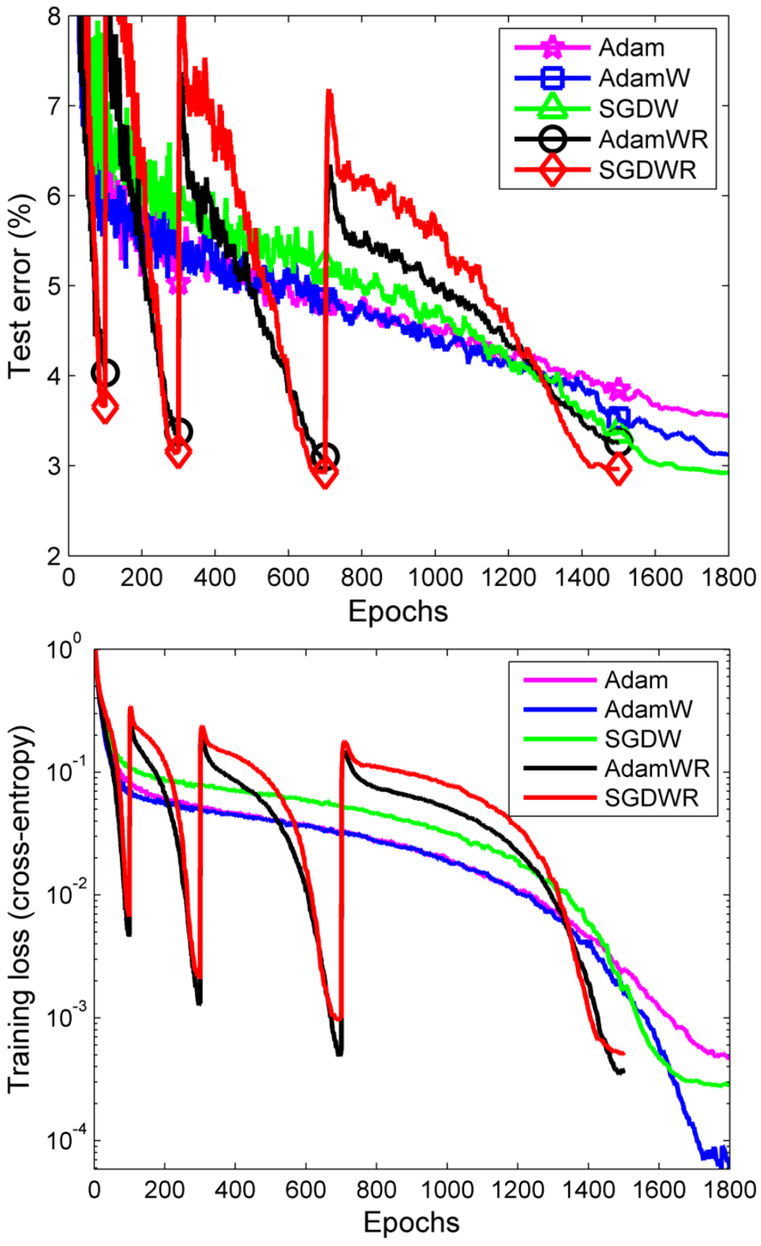
SuppFigure 4: Learning curves (top row) and generalization results (Top-5 errors in bottom row) obtained by a 26 2x96d ResNet trained with Adam and AdamW on ImageNet32x32.



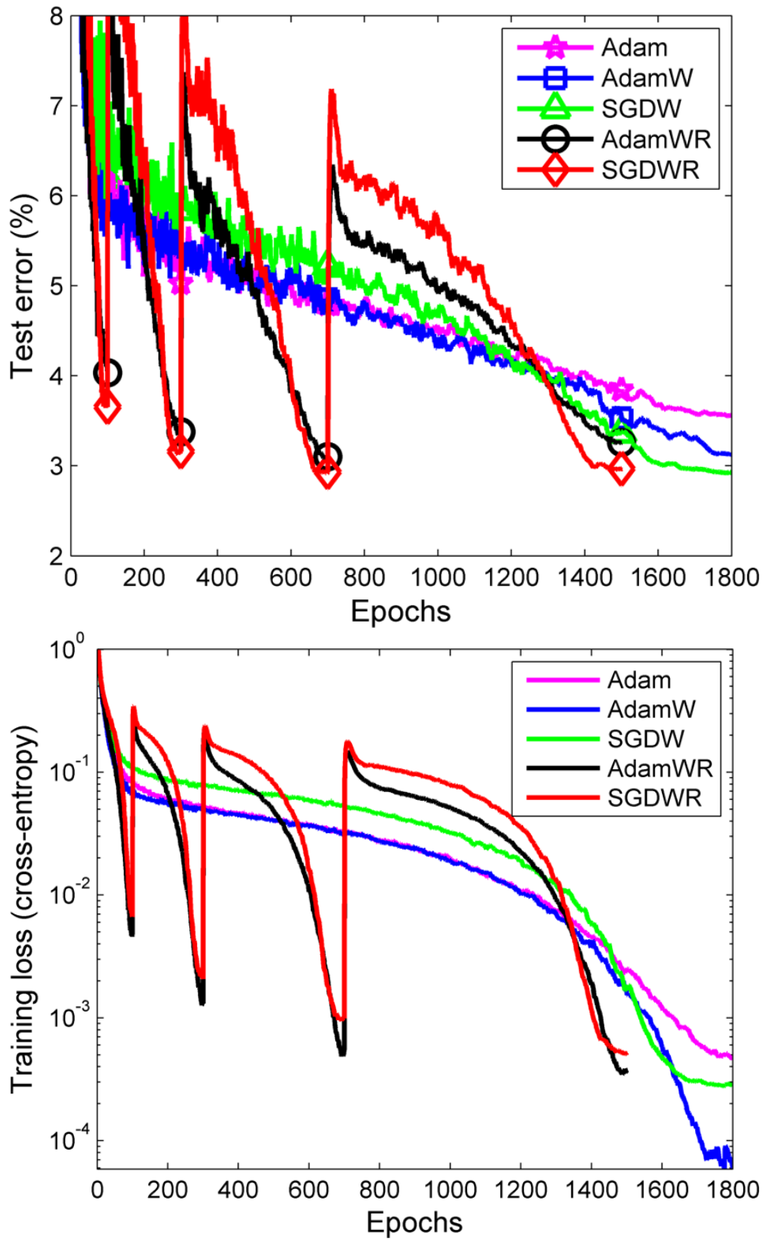


Sup图4：学习曲线（上排）与泛化性能结果（下排为Top-5错误率）。)

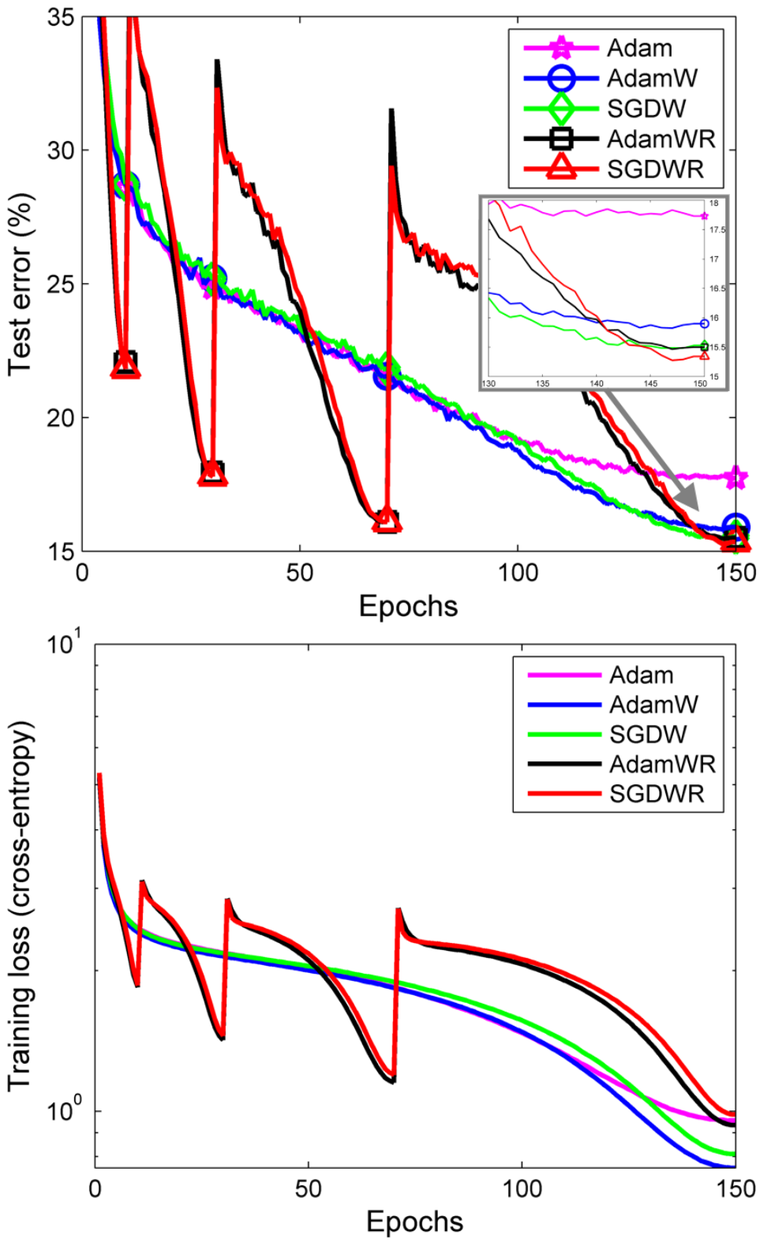
obt这些结果是通过在ImageNet32x32数据集上使用Adam优化器及AdamW算法训练的26层2x96维度的ResNet网络获得的。



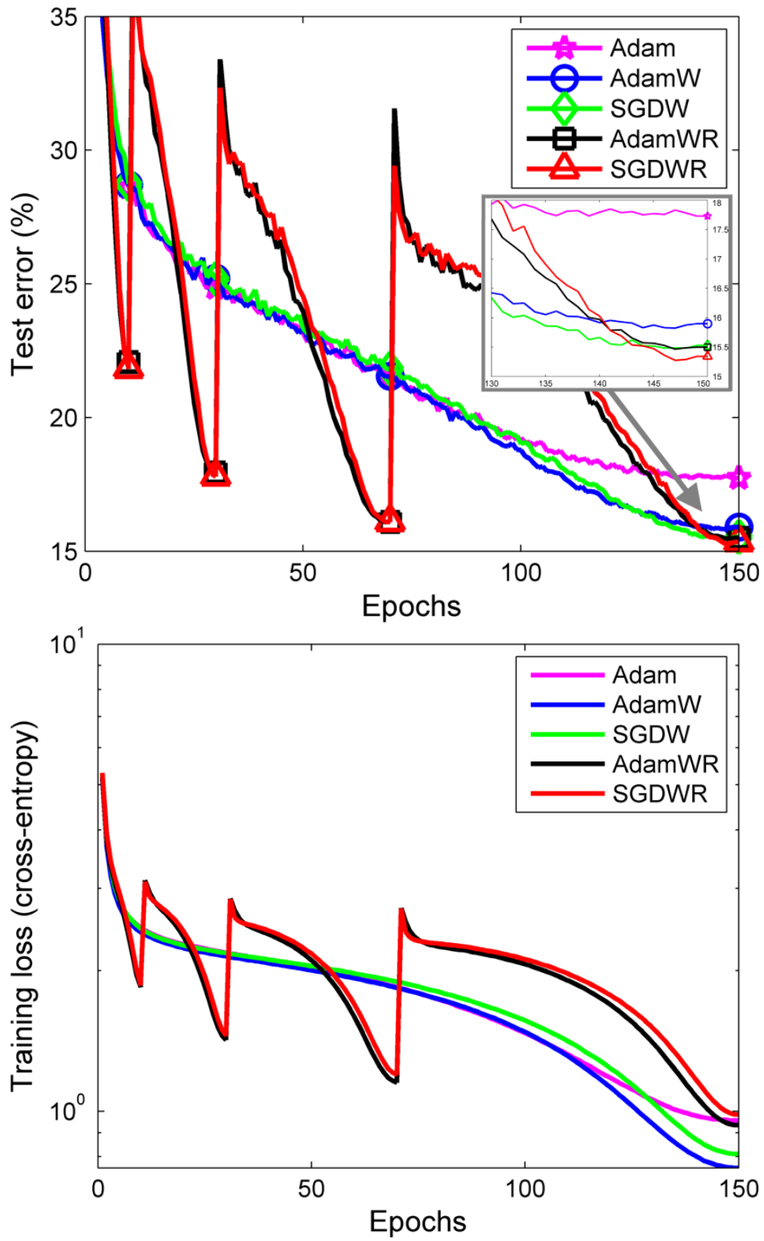
SuppFigure 5: Test error curves (top row) and training loss curves (bottom row) for CIFAR-10.



补充图5：CIFAR-10数据集的测试误差曲线（上行）与训练损失曲线（下行）。



SuppFigure 6: Test error curves (top row) and training loss curves (bottom row) for ImageNet32x32.



补充图6：Imagenet32x32的测试误差曲线（上排）与训练损失曲线（下排）。